

Computer Science =

* números Binarios =

El sistema Binario Solo tiene 2 números. Tiene

0 y un 1. Esto se debe a cómo funcionan

los ordenadores a un nivel básico. Solo son capaces de leer si un interruptor está encendido o apagado.

↓
0

↓
1

Este sistema Binario significa que el sistema

numérico funciona en "Base 2" en lugar de

base 10. Esto significa que sube en orden de

2 en 2.

Comparamos el Sistema decimal con el Binario.

decimal

1	5	7	8	4	8	3
10^6	10^5	10^4	10^3	10^2	10^1	10^0
1.000.000	100.000	10.000	1000	100	10	1

cada nueva posición es un orden 10. Al

Final sumamos el número superior multiplicado

por su posición

$$\text{Tenemos} = 1.000.000 + 500.000 + 70.000 + 8.000 + 400 + 80 + 3 = 1.578.483$$

Binario

1	0	1	1	1	1	1
2^6	2^5	2^4	2^3	2^2	2^1	2^0
64	32	16	8	4	2	1

$$64 + 0 + 16 + 8 + 4 + 2 + 1 = 95$$

Ejercicio = Binario a decimal

A) 10110011 =

1	0	1	1	0	0	1	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

* $128 + 32 + 16 + 2 + 1 = 179$ decimal

B) 00111001 =

0	0	1	1	1	0	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

$32 + 16 + 8 + 1 = 57$ decimal

Ejercicio = Decimal a Binario

A) 296 =

0	1	0	0	1	0	1	0	0	0
2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
512	256	128	64	32	16	8	4	2	1

296	40	8
-256	-32	-8
* 40	* 8	* 0

Si podemos restarlo le colo como un 1.

El resultado es = 100101000

Questionario 1 = números Binarios

A) 45 =

0	1	0	1	1	0	1
2^6	2^5	2^4	2^3	2^2	2^1	2^0
64	32	16	8	4	2	1

$$\begin{array}{r} 45 \\ - 32 \\ \hline * 13 \end{array} \quad \begin{array}{r} 13 \\ - 8 \\ \hline * 5 \end{array} \quad \begin{array}{r} 5 \\ - 4 \\ \hline * 1 \end{array} \quad \begin{array}{r} 1 \\ - 1 \\ \hline * 0 \end{array} = 101101$$

$$* 32 + 8 + 4 + 1 = 45$$

B) 1011 =

1	0	1	1
2^3	2^2	2^1	2^0
8	4	2	1

$$* 8 + 2 + 1 = 11$$

C) 100110 = 38

1	0	0	1	1	0
2^5	2^4	2^3	2^2	2^1	2^0
32	16	8	4	2	1
32 + 4 + 2 = 38					

Seccion 2Análisis de Algoritmos =

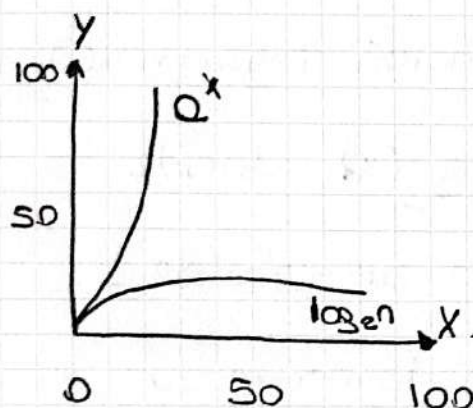
Funcion Logoritmica = Una Función logoritmica tiene la Forma de $g(x) = \log_Q(x)$, donde Q es una constante positivo distinto de 1.

El Crecimiento de la Funcion logoritmica crece de manera muy lenta a medida que x aumenta.

Funcion Exponencial = La Funcion Exponencial $F(x) = Q^x$, donde Q es una constante positivo, distinto de 1.

El Crecimiento de la Funcion Exponencial crece de manera exponencial.

Ejemplo = Tenemos un algoritmo en donde hoy tiempos de ejecución. En "X" se insertan datos y en el eje "Y" cuanto tiempo nos va a tomar



En la "función logarítmica" podemos insertar 100 datos y obtenemos un tiempo por debajo de 5. ↓ ejecución

En la "F. Exponencial", sólo insertamos 5 datos y el tiempo de ejecución crece superando los 100.

$$E1 = \log_x (y) = b$$

$$x^b = y$$

$$2^b = 8$$

$$2^3 = 8$$

$$E2 = \log_2 (64) = 6$$

$$2^n = 64$$

$$2^6 = 64$$

* Podemos introducir tantos datos y nuestro algoritmo se convierte en una línea horizontal,

n-Notación =

n-Notación se representa como una función de n . Esto significa simplemente que el programa va a estar relacionado con la variable n .

La notación n ; no es un análisis de cuánto tardará en ejecutarse un algoritmo, sino un análisis de cómo se escalará (aumentará) el algoritmo con más y más datos de entrada.

* La notación " n " describe el rendimiento o la complejidad de un algoritmo en función del tamaño de la entrada n .

Indica, cómo el tiempo de ejecución o el uso de memoria del algoritmo crece a medida que crece n .

Ejemplo 1=

A) Input $\boxed{A}$ Output
 $=$ n $=$ n

B) Input $\boxed{B}$ Output
 $=$ n $=$ n^2

Comparamos los algoritmos A y B para saber ¿Cuál de estos algoritmos (A y B) es el más rápido?

A) Input $\boxed{A}$ Output
100 100

B) Input $\boxed{B}$ Output
100 10.000

* $A \geq B$. A es mejor que B porque utiliza menos tiempo que B (nº. de operaciones)

Ejemplo 2=

Cada ciclo de un programa dura 0,001

Segundos. ¿cuanto más rápido tardará el

$n \cdot \log(n)$ que el n^2 ? Si se introducen 1.000 datos. (n)

- $n \log n = 1000 \log(1000) = 9960 \cdot 0.01 = 9.96''$ Segundos

- $n^2 = 1000^2 = 1.000.000 \cdot 0.01 = 1.000$ Segundos = 16'

* tenemos que encontrar el tipo de estructura de datos & técnicas para crear el algoritmo/s más eficiente & eficientes posibles.

Notación Big O:

La notación Big O es una forma de describir el rendimiento o la complejidad de un algoritmo en términos de: cantidad de trabajo que realiza, en función del tamaño de su entrada.

Se utiliza para clasificar algoritmos según cómo su tiempo de ejecución y el uso de espacio (memoria) crece a medida que el tamaño de la entrada aumenta. (n)

La notación Big O proporciona una forma de expresar el peor caso posible para el tiempo de ejecución del algoritmo.

Ejemplos Comunes de notación Big O =

- $O(1)$ = Tiempo Constante, el tiempo de ejecución no cambia con el tamaño de la entrada.
- $O(\log n)$ = Tiempo logarítmico, el tiempo de ejecución crece logarítmicamente con el tamaño de la entrada.
- $O(n)$ = Tiempo Lineal, el tiempo de ejecución crece linealmente con el tamaño de entrada.
- $O(n \log n)$ = Tiempo Lineal-logarítmico, común en algoritmos de ordenamiento eficientes como el Merge Sort.
- $O(n^2)$ = Tiempo cuadrático, el tiempo de ejecución crece proporcionalmente al cuadrado del tamaño de la entrada, común en algoritmos de Fuerza Bruta.
- $O(2^n)$ = Tiempo exponencial, el tiempo de ejecución se duplica con cada aumento en el tamaño de la entrada, común en algoritmos recursivos.
- $O(n!)$ = Tiempo Factorial, el tiempo de ejecución crece factorialmente con el tamaño de la entrada, común en problemas de permutación.

La notación Big O se centra en el comportamiento asintótico del algoritmo, lo que significa que se enfoca en cómo el algoritmo se comporta para entradas muy grandes.

Comparación de Algoritmos: Podemos comparar algoritmos y determinar cuál es más eficiente en términos de tiempo de ejecución o uso de memoria. Un algoritmo $O(n)$ será más eficiente que un algoritmo $O(n^2)$ para grandes entradas.

* Cuanto más nos acerquemos a O , más rápido será nuestro algoritmo.

Notación Big representa un límite =

$O(n^3)$

+
Rápido que n^2

* $O(n \log n)$

En el peor de los casos pero
 $n \cdot \log n$.

* $\Theta(n)$

igual que n

$\Omega(n^2)$

Lento o igual que " n^2 " (en el mejor
de los casos)

$\omega(n^3)$

más Lento que n^3

A) Ejercicio =

Un programa se ejecuta a 1 segundo por " n ".

¿Cuánto más tardará un programa que se ejecuta
en n^3 , que un programa que se ejecuta en
 n^2 si $n = 25$? (n mayor - n menor)

* Para resolver, vamos a calcular cuánto tiempo
tomará cada programa para $n = 25$ y luego
encontrar la diferencia entre los dos tiempos
de ejecución.

* Calcular el tiempo de ejecución para cada programa,

1. Programa que corre en $n^2 =$

$$(25)^2 = 625 \text{ segundos}$$

2. Programa que corre en $n^3 =$

$$(25)^3 = 15625 \text{ segundos}$$

* Encontrar la diferencia entre los tiempos de ejecución.

$$15625 - 625 = 15.000 \text{ segundos}$$

La notación Big O se utiliza para describir la complejidad del tiempo de ejecución de los algoritmos, en términos del tamaño de la entrada n .

Podemos ver que el algoritmo cúbico (n^3) toma significativamente más tiempo que el algoritmo cuadrático (n^2) para una entrada del mismo tamaño.

B) ¿Qué programa es más rápido en general?

Programa A:

Programa B:

Load: $O(n)$

Load: $O(1)$

Operar: $O(n)$

operar: $O(1)$

Salida: $O(n)$

Salida: $O(n^2)$

Para determinar cuál de los dos programas es más rápido en general, debemos analizar lo, "Complejidad total" de cada uno combinando las complejidades de las diferentes etapas (Load, Operar, Salida)

Análisis de Complejidad total:

* Programa A:

Load = $O(n)$

Operar = $O(n)$

Salida = $O(n)$

Para combinar estas complejidades, sumamos las complejidades de cada etapa.

$$O(n) + O(n) + O(n) = O(3n)$$

En notación Big O, eliminamos las constantes,

por lo que lo, Complejidad total del Programa A

es = $O(n)$

Programa B =

Load = $O(1)$

Operar = $O(1)$

Salida = $O(n^2)$

Sumamos las complejidades de cada etapa.

$$O(1) + O(1) + O(n^2) = O(2) + O(n^2)$$

En la notación Big O, nos centramos en el término dominante y eliminamos constantes. ($O(n^2)$).

● Comparación de complejidades:

Programa A = $O(n)$

Programa B = $O(n^2)$

Un Algoritmo con complejidad $O(n)$ es más

eficiente que un algoritmo con complejidad $O(n^2)$

● para valores grandes de n . Esto se debe a

que la complejidad cuadrática $O(n^2)$ crece

mucho más rápido que la complejidad lineal

$O(n)$ a medida que aumenta el tamaño de

la entrada. "El programa A es en general más

rápido que Programa B."

La notación "Big O" se utiliza para describir la eficiencia de cada etapa de los programas en términos del tamaño de la entrada n .

- $O(1)$ = tiempo constante, independiente del tamaño de la entrada.
- $O(n)$ = Tiempo Lineal, crece proporcionalmente al cuadrado del tamaño de la entrada.
- $O(n^2)$ = Tiempo Cuadrático, crece proporcionalmente al cuadrado del tamaño de la entrada.

c) ¿Qué letra griega es lo más importante para comparar algoritmos?

La letra griega para comparar algoritmos es "O" es fundamental en la teoría de la complejidad computacional porque describe la eficiencia de un algoritmo en términos del peor caso posible.

Permite comparar cómo se escala el tiempo de ejecución y el uso de recursos de diferentes

Algoritmos a medida que aumenta el tamaño de la entrada.

Ejemplo = Representación de Algoritmo que realiza una búsqueda en una lista de datos utilizando bucles anidados.

- Bucle Exterior =

Dado que hay un Bucle Anidado que itera sobre todos los elementos de la lista, la complejidad del tiempo de ejecución del algoritmo es $O(n^2)$, donde "n" es el número de elementos en la lista.

- La imagen muestra cómo se realiza la comparación de cada elemento "N" con todos los elementos "W" en la lista.

- Se cuenta cada comparación, y el resultado es que para una lista de tamaño $n=4$, se realizan $4 \times 4 = 16$ comparaciones, lo que es $O(n^2)$.

Sección 3Arroys=

¿como se almacenan los datos?

- Los estructuras de datos son la forma de tomar datos, organizarlos y poder acceder a ellos y guardarlos de determinadas formas que se ajusten a nuestro objetivo.

3	4
0x00	0x01

var x = 3 ; *var y = 4
↓

*var y = 4

0x01

Estamos asociando
a y → 0x01

Estamos asociando x
con el x → 0x00

Cada vez que el usuario
llame a x, vamos a
obtener lo que este en
la ubicación "0x00"

$$x + y = 0x00 + 0x01$$

$$3 + 4 = 7$$

0x00 (direccion)

3	2	6	1
0	1	2	3

Direccion principal es 0x00. Utilizamos los mini nombres para encontrar alguna pieza de datos, esto se lo conoce como un array.

→ 0x00

var x = [3, 2, 6, 1]

x[1] = 2

* Si queremos retornar "x[1]". El procedimiento para encontrar "x[1]" es: 0x00, luego el nombre que es 1 y va a devolver el 2.

Fixed Array

- Matriz = Fija =

¿Qué es una matriz?

Una matriz es una estructura de datos que almacena elementos del mismo tipo en posiciones contiguas de memoria, accesibles mediante índices numéricos.

Un Array por definición, es una estructura de datos que almacena una colección de elementos del mismo tipo en posiciones de memoria contiguas.

7	8	9	10	1	2	3
0	1	2	3	4	5	6

Ejemplo = Crear una matriz de tres números.

$x = [1, 2, 3] \rightarrow$

1	2	3
---	---	---

La matriz empieza en cero 0 por la indexación cero y además en el final se suele utilizar $n-1$.

Los matrices fijas se crean con un tamaño fijo y las mantienen durante toda su ejecución.

mivor[0] 5.6
mivor[1] 4.5
mivor[2] 3.3
mivor[3] 13.2
mivor[4] 4.0
mivor[5] 34.33

Un array funciona designando una sección de memoria para su almacenamiento. Cada vez que llamamos a la matriz, devuelve la ubicación de inicio de la matriz.

Tiempos de Ejecución de operaciones (Array Fijo) =

- Insertion Aleatoria = Significa: insertar un elemento en una posición aleatoria.

- Complejidad = $O(n)$ porque puede requerir mover todos los elementos para hacer espacio para el nuevo elemento.

- Inserción hacia Adelante: Significa: insertar un elemento al principio del array.

. Complejidad = $O(n)$ Lineal, porque se deben mover todos los elementos existentes un lugar hacia atrás.

- Inserción hacia atrás = Significa: insertar un elemento al final del array.

. Complejidad = $O(1)$ Constante, ya que no se necesita mover ningún elemento, simplemente se agrega al final.

- Borrado hacia Adelante = Significa: eliminar el primer elemento del array.

. Complejidad = $O(n)$ Lineal, porque se deben mover todos los elementos para llenar el espacio.

- Borrado hacia atrás = Significa: eliminar el último elemento del array.

. Complejidad = $O(1)$ Constante,
ya que se elimina el último
elemento sin necesidad de
mover otros elementos.

- Borrado Aleatorio = Borrar un elemento en una posición
aleatoria del array

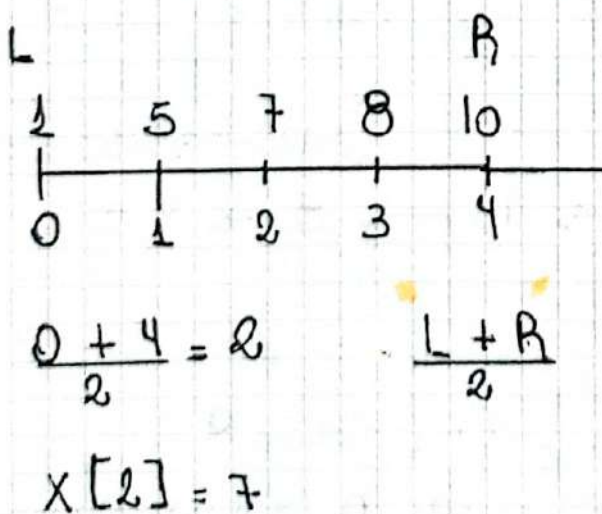
. Complejidad = $O(n)$ lineal.
Borrar aleatoriamente
de la matriz puede hacerse
en tiempo $O(1)$ si simplemente
se elimina dejando un agujero.
Sin embargo, si se desea eliminar
un elemento y no dejar un
agujero, es necesario desplazar
todos los elementos posteriores
hacia adelante para cubrir
el agujero, lo que toma
tiempo lineal en el peor de los
casos.

- Buscon sin clasificar = Significado = Buscon un elemento en un array no ordenado.

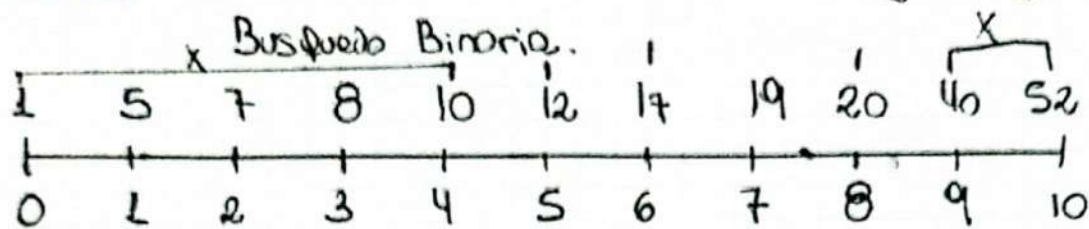
• Complejidad = $O(n)$ lineal, en el peor de los casos, se debe verificar cada elemento del array para encontrar el buscado. Esto toma tiempo lineal.

- Buscon (Busqueda Ordenada) = Significado = Buscon un elemento en un array ordenado.

• Complejidad = $O(\log n)$, logaritmo natural de n , utilizando la busqueda binaria, se puede encontrar un elemento en un array ordenado en tiempo logaritmo, dividiendo repetidamente el espacio de busqueda a la mitad.



Ejercicio = Encontrar el num. 19 del array utilizando



$$0 + 10 / 2 = 5$$

$$x[5] = 12$$

$$6 + 10 / 2 = 8$$

$$x[8] = 20$$

$6 + 7 / 2 = 6.5$ * como tiene coma, se tiene que

$$x[6] = 17$$

tomar el num. entero solamente.

seria 6.

$$7 + 7 / 2 = 7$$

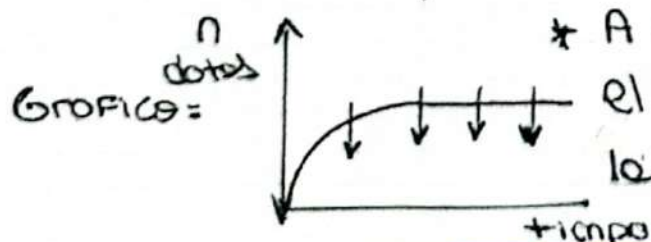
$$x[7] = 19$$

$$512 \rightarrow 256 \rightarrow$$

$$\text{Ejemplo} = 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2$$

$$\rightarrow 1$$

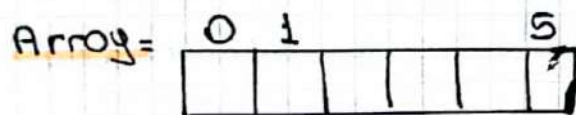
$$128 = 7 \text{ (pases)} ; 256 = 8 ; 512 = 9$$



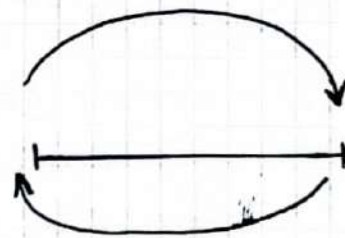
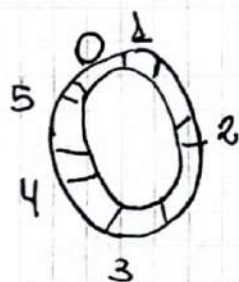
* A medida que n (dots) aumenta el tiempo de ejecución toma la forma de el gráfico del $\log(n)$.

- Matriz Circular =

Una Matriz Circular es una estructura de datos que utiliza un array de tamaño fijo y trata sus posiciones como si estuvieran conectadas de forma circular.



Array Circular = Ejemplo ilustrativo.



Esto significa que, después del último elemento del array, el siguiente elemento está en la primera posición.

Características =

1. Conexion Circular = La última posición del array está conectada a la primera posición, formando un círculo.

2. Indices Modulares = Los índices se gestionan usando aritmética modular para asegurar que se "envuelven" alrededor del final del array hacia el comienzo.

Ejemplo $\begin{cases} n = \text{tamaño del array} \\ i = \text{es un índice} \end{cases}$

Siguiente índice = $(i + 1) \% n$

Índice anterior = $(i - 1 + n) \% n$

3. Cola Circular = Una cola circular es una implementación común de una matriz circular, donde se utiliza para gestionar de manera eficiente el acceso y almacenamiento de datos en una estructura FIFO (First in, First Out).

Ejemplo =

Imagine un banco; la persona que se por es la primera atendida.

Describe un método de organización de datos en el que el primer elemento en agregarse a una E.D.D. es el primero en ser eliminado.

4 Buffer Circular = Imagina una Cola donde los elementos se van añadiendo y eliminando de forma continua. Esta Cola tiene un tamaño fijo. Cuando se llene, el siguiente elemento que se añade sobrescribe al primero y así sucesivamente.

Ventajas de una Matriz Circular:

. Uso eficiente del espacio = Se evita la necesidad de mover elementos cuando se añaden o eliminan elementos al principio o al final.

. Tiempo Constante para Operaciones = Operaciones como insertar o eliminar elementos al principio o al final pueden realizarse en tiempo constante $O(1)$.

- Matrices Dinomicas :

Un "Arroz Dinomico" puede crecer o reducir su tamaño durante la ejecución de un programa. Esto significa que no necesitamos definir su tamaño al principio, sino que podemos ajustarlo según las necesidades.

Los "elementos" de la matriz dinomica se almacenan de forma contigua, al principio

de la matriz subyacente, y las posiciones restantes hacia el final de la matriz

subyacente se reservan o no se utilizan.

Se pueden "agregar elementos" al final de la matriz dinomica en tiempo constante $O(1)$

utilizando el espacio reservado, hasta que este espacio se consume por completo.

Cuando "todo el espacio" se ha consumido y se desea agregar un elemento adicional, es necesario aumentar el tamaño de la matriz de tamaño fijo.

Los elementos pueden eliminarse del final de una matriz dinamica en tiempo constante, ya que no es necesario cambiar su tamaño.

El número de elementos utilizados por el contenido de la matriz dinamica es su tamaño lógico & tamaño, mientras que el tamaño de la matriz se denomina capacidad & tamaño físico de la matriz dinamica.

(*) - indice	inicio	Fin	medio
A) $O(n)$ Linealmente	$O(1)$	$O(1)$ Final conocido $O(n)$ Final desconocido Elementos	$O(n)$
B) $O(1)$ Constante	—	—	—
C) $O(1)$ Constante	$O(n)$	$O(1)$ A. Amortizado	$O(n)$
Exceso de espacio medio A = $O(n)$, B = O , C = $O(n)$	A = Lista Enlazada B = Array C = Array Dinamico		

- Análisis Amortizado = Es un método para analizar la complejidad de un algoritmo determinado, es decir, la cantidad de un recurso, especialmente tiempo o memoria, que se necesita para ejecutarlo. La motivación del Análisis Amortizado es que considerar el tiempo de ejecución en el peor de los casos puede ser pesimista, en su lugar, 'el Análisis Amortizado promedia los tiempos de ejecución de las operaciones'.

- Tiempo de ejecución =

$O(n)$: Significa que el tiempo que tarda un algoritmo en ejecutarse crece linealmente con el tamaño de la entrada (n). Si duplicamos el tamaño de la matriz, aproximadamente duplicaremos el tiempo de ejecución.

$O(1)$: Significa que el tiempo de ejecución es constante, independientemente del tamaño de la entrada (n). El tiempo que tarda el algoritmo

no varía significativamente al aumentar el tamaño de la matriz.

A). Acceso a un elemento = (Cuadro $\otimes$)

- Caso General = Acceder a un elemento en una posición específica de una matriz dinámica suele ser una operación de $O(1)$, ya que se puede calcular directamente su dirección de memoria.

- Caso Particular = Si la matriz está implementada de forma que los elementos no estén almacenados de forma contigua (Lista Enlazada), el acceso suele ser más lento, incluso $O(n)$ en el peor de los casos.

B). Insertión al Final = Si hay espacio disponible al final de la matriz, insertar un nuevo elemento suele ser $O(1)$, ya que solo se necesita actualizar el índice del último elemento.

c). Inserción en el medio: Insertar un elemento en el medio (matriz D.) implica desplazar todas las elementos posteriores una posición hacia la derecha, es una operación de $O(n)$, el tiempo de ejecución crece proporcionalmente al n^2 de elementos que hay que mover. ◀

d). Eliminación: Eliminar un elemento en el medio suele ser $O(n)$ ya que hay que desplazar los elementos posteriores.

- Aproximación = Los Aproximaciones suelen sugerir varias noiones.

1.. Complejidad Computacional = operaciones como redimensionar & insertar elementos en el medio, pueden ser costosas en términos de tiempo de ejecución. (Operaciones costosas).

Estimaciones a menudo se utilizan para estimar el tiempo que toma una operación.

2.. Espacio en memoria = Las M.D. se obtienen en la memoria de forma dinámica, el sistema operativo asigna y libera memoria según sea necesario.

Estimaciones; para evitar agotar la memoria, se realizan "estimaciones" sobre el tamaño máximo que podría alcanzar una matriz.

3. Precision = Dots Aproximados, en gráficos por computadora, se pueden utilizar aproximaciones para acelerar el renderizado.

Tipos de Aproximaciones =

A) Aproximaciones Asintóticas =

- Notación O grande = Se utiliza para describir la tasa de crecimiento de una función en notación asintótica. Por ejemplo, $O(n)$ indica que el tiempo de ejecución crece linealmente con el tamaño de la entrada.
- Simplificaciones = Se desprecian términos de menor orden para obtener una expresión más sencilla.

Quiz ArrayS =

1) Si tenemos un Array Circular =

$\text{cArray} = [0, 4, 3, -, -, -]$ (donde $-$ es igual

a espacio vacío), qué aspecto tendría el array

si realizamos una inserción en la parte

del ^{→ (Front)}antero con el número 8.

$[0, 4, 3, -, -, -]$ Array Original.



insertion 8

$[0, 4, 3, -, -, 8]$ Array después de la inserción.

Para insertar un elemento =

El tamaño del array es 6.

• Identificar el índice del Frente = El índice del Frente es 0

• Determinar el índice del siguiente elemento = Se calcula como $= (\text{Front} + 1) \% \text{Size}$ (Size = tamaño de Array).

• Verificar

2) Si tuviéramos una Matriz Circular = [5, 4, 3, -, -] (donde "-" es igual a espacio vacío) el que índice estaría apuntando el cursor de inicio después de un borrado desde el Frente. Supongamos que el cursor está apuntando actualmente al índice 0.

- Matriz Circular = Es una estructura de datos en la que el último elemento está conectado al primero, formando un círculo.
- cursor de inicio = Indica el primer elemento de la matriz.
- Borrado desde el Frente = Eliminamos el elemento al que apunta el cursor de inicio.

[5, 4, 3, -, -] Antes del Borrado.

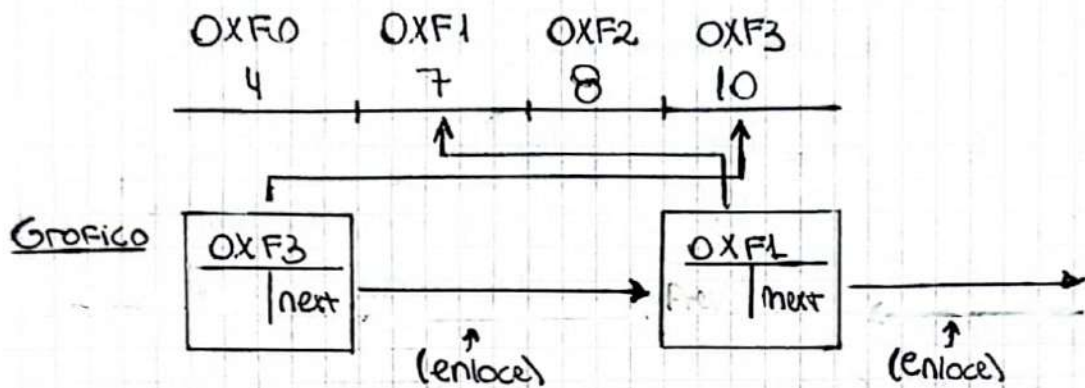
[-, 4, 3, -, -] Después del Borrado.

- Al borrar un elemento desde el frente de una matriz, el cursor de inicio avanza una posición al índice 1, que en el ejemplo contiene el 4.

Seccion 4

Listo Enlazado

Model =



Un nodo tiene al menos dos partes =

Datos = Se Almacena la información que queramos representar en la lista.

Enlace o puntero = Es una referencia al siguiente nodo de la lista. Este enlace es lo que conecta los nodos entre si, formando la estructura enlazada.

Ejemplo =

imaginar lista de compra

Este "nodo" contendría

información como "nombre"

y la "cantidad". Además, cada nodo tendría

un "puntero" o "enlace" que apuntaría al siguiente

artículo de la lista. (el siguiente nodo).

- Arroz = 2 paquetes
- Solso = 1 paquete
- Manzana = 2 cajas

Características de nodos

- Flexibilidad = Los Listos Enlazados son muy Flexibles para agregar o eliminar elementos, ya que solo necesitamos modificar los enlaces de los nodos adyacentes.
- Tamaño Dinámico = No necesitan un tamaño fijo como los arrays, pueden crecer o encoger según sea necesario.
- inserción en cualquier posición = Podemos insertar un nuevo elemento en cualquier lugar de la lista, no solo al final.

Lista Enlazada = Una Lista Enlazada es una colección de elementos conectados entre sí, donde cada elemento (nodo) guarda un dato y una referencia al siguiente elemento.

Ejemplo = Cada Voz es un nodo = Cada nodo contiene:

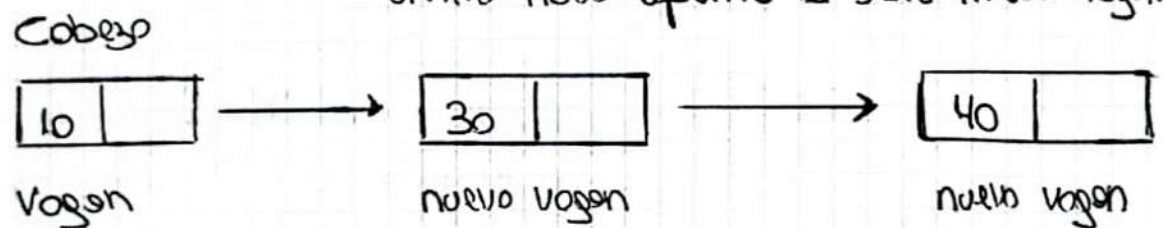
- un "dato" (lo que queramos almacenar, número, palabra, etc) y un "puntero" que apunte al siguiente nodo.

.El primer Vagón es la Cabeza: punto de entrada.

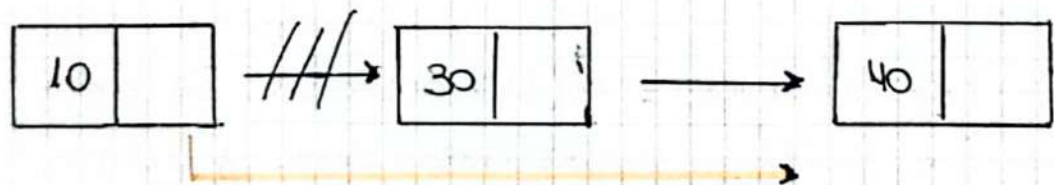
.El último Vagón no apunta a ningún otro: Indica el Fin de la lista.

Funcionamiento =

.Agregar un nodo = Para agregar un nuevo nodo, creamos un nuevo vagón, lo llenamos con el dato y hacemos que el puntero del último nodo apunte a este nuevo vagón.



.Eliminar un nodo = Para eliminar un nodo, hacemos que el puntero del nodo anterior apunte al nodo siguiente.



$O(n)$ Temporal lineal =
Esto significa que el tiempo que tarda un algoritmo en ejecutarse crece directamente proporcional al tamaño de la entrada (n).

$O(1)$ Temporal Constante
Significa que el tiempo de ejecución siempre es el mismo, independientemente del tamaño de la entrada.

- caso promedio = $O(n)$ lineal. Para encontrar un elemento, hay que recorrer la lista desde el principio hasta encontrarlo, lo que puede tomar " n " pasos, siendo n el número de elementos en la lista.
- mejor caso = $O(1)$ Constante. El elemento a buscar es el primero, se encuentra en un solo paso.

B) Inserción al Inicio = $O(1)$ Constante. Se crea un nuevo nodo y se hace que apunte al nodo actual de la cabeza. La cabeza de la lista (Head) se actualiza para apuntar al nuevo nodo.

Lista Enlazada. Tiempos de ejecución =

En los Listas Enlazadas el tiempo de acceso depende de la posición del elemento.

A). Acceso a un elemento =

- caso promedio = $O(n)$ lineal. Para encontrar un elemento, hay que recorrer la lista desde el principio hasta encontrarlo, lo que puede tomar " n " pasos, siendo n el número de elementos en la lista.
- mejor caso = $O(1)$ Constante. El elemento a buscar es el primero, se encuentra en un solo paso.

B) Inserción al inicio = $O(1)$ constante. Se crea un nuevo nodo y se hace que apunte al nodo actual de la cabeza. La cabeza de la lista (Head) se actualiza para apuntar al nuevo nodo.

c) Inserción al Final = $O(n)$ lineal. Es necesario recorrer toda la lista hasta encontrar el último nodo para enlazar el nuevo nodo.

d) Inserción posición arbitraria = $O(n)$ lineal. Se debe recorrer la lista hasta encontrar la posición donde se quiere insertar el nuevo nodo.

e) Eliminación del primer elemento = $O(1)$ constante. Se actualiza la cabeza de la lista para apuntar al segundo nodo.

f) Eliminación del último elemento = $O(n)$ Lineal. Se debe recorrer la lista hasta encontrar el penúltimo nodo para actualizar su enlace. (antes del último).

g) Eliminación de un elemento en una posición arbitraria: $O(n)$ Lineal. Se debe recorrer la lista hasta encontrar el nodo a eliminar y luego actualizar los enlaces de los nodos adyacentes. (muy próxima a otro caso).

Cuadro =

Operación	TP	MC	PC
Acceso	$O(n)$	$O(1)$	$O(n)$
I. Inicio	$O(1)$	$O(1)$	$O(1)$
I. Final	$O(n)$	$O(n)$	$O(n)$
I. Pos. A.	$O(n)$	$O(n)$	$O(n)$
E. P.E	$O(1)$	$O(1)$	$O(1)$
E. U.E	$O(n)$	$O(n)$	$O(n)$
E. Pos. A.	$O(n)$	$O(n)$	$O(n)$

¿Por qué estos tiempos?

• Acceso = Al no tener un índice como los arrays, se debe recorrer la lista secuencialmente para encontrar un elemento.

• Inserción/eliminación al inicio = Estas operaciones son rápidas porque solo involucran modificar el puntero de la cabeza. $O(1)$

• Inserción/eliminación en otras posiciones = Requieren recorrer la lista para encontrar la posición correcta y actualizar los enlaces. $O(n)$

Ejemplo =

Constante $O(1)$ = Acceden al primer elemento de un array siempre toma el mismo tiempo, sin importar cuántos elementos haya.

• Lineal $O(n)$ = Buscar un elemento en una lista enlazada no ordenada requiere recorrer toda la lista, lo que toma más tiempo a medida que la lista crece.

(: Ejemplo Lista Enlazada (Agregar un nuevo nodo al principio de la lista)

1. Clase Node =

• Representa un nodo individual en la lista.

• Tiene dos atributos:

- data = Almacena el dato del nodo
- next = Un puntero que apunta al siguiente nodo en la lista.

2. Clase LinkedList =

A). Represento la lista enlazada completa

B). Tiene un atributo:

• head = un puntero al primer nodo de la lista.

C). El método "--init--" inicializa la lista estableciendo "head" como "None", indicando que inicialmente la lista está vacía.

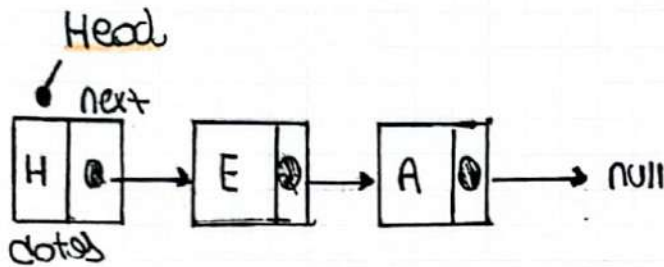
D). El método "add" agrega un nuevo nodo al principio de la lista:

- Crea un nuevo nodo con el dato proporcionado.
- hace que "next" del nuevo nodo apunte al nodo...

... fue era lo Cabezal antes. (el primer nodo).

e) Actualizo el "head" para que apunte al nuevo nodo, convirtiendolo en el nuevo primer nodo.

Grafico Lista Enlazada:



El primer nodo se denomina "cabezal" de la lista enlazada. Es el nodo al que siempre apuntamos cuando llamamos a nuestra lista enlazada.

Los tiempos de búsqueda de las listas enlazadas suelen ser más lentos que los de una matriz, incluso si la lista ~~no~~ está ordenada, ya que no hay forma de saltar a un punto determinado.

D = Dato

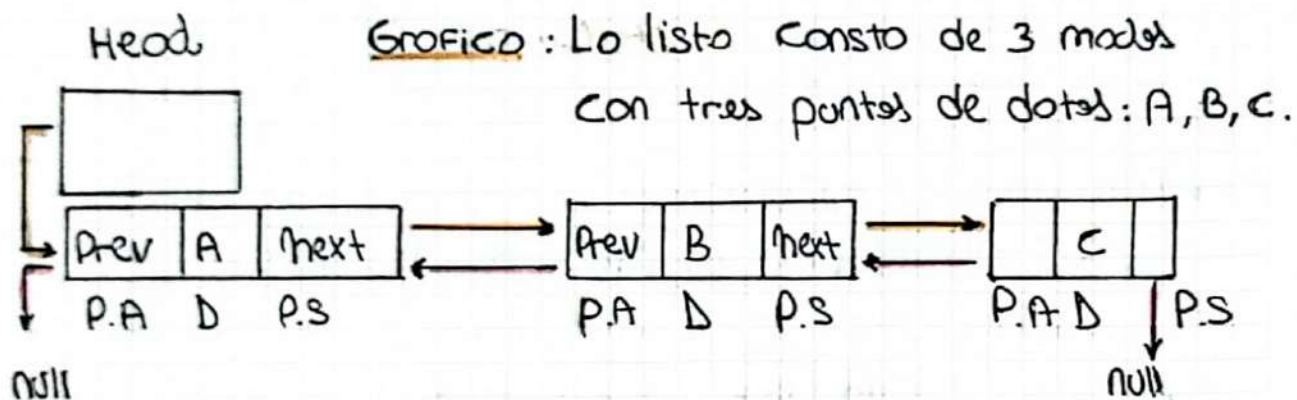
P.S = Puntero
al nodo
siguiente

P.A = Puntero al
nodo anterior

Listas Enlazadas Dobles:

Una Lista Enlazada doble
de datos lineal f.e., o. c.

Lista enlazada Doble, permite el acceso a
los elementos tanto en orden ascendente
como descendente.



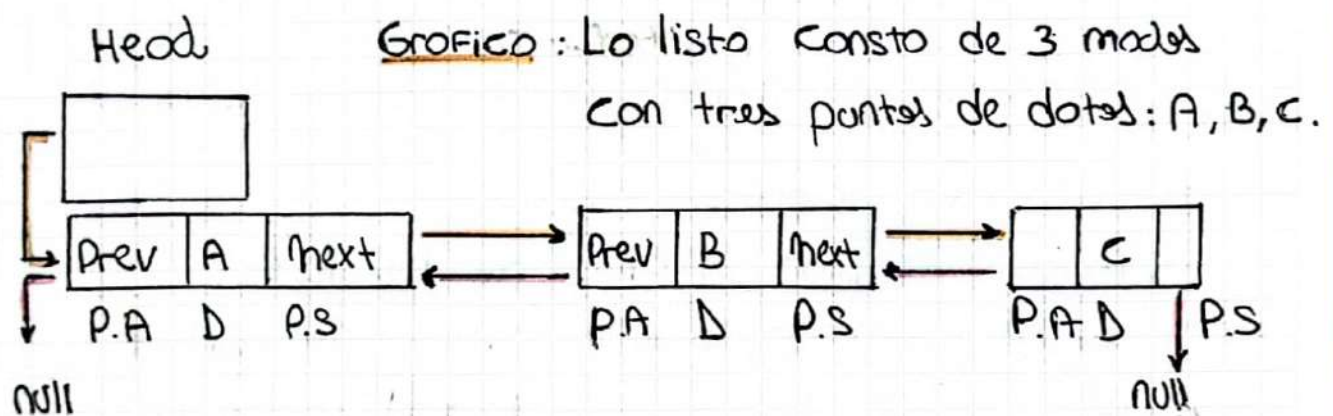
Estructura de un nodo:

Cada nodo en una lista enlazada doble
contiene tres campos:

- . Dato = El valor que almacena en el nodo.
- . Puntero al siguiente nodo: Un puntero que
apunta al siguiente
nodo en la lista.
- . Puntero al nodo anterior: Un puntero que apunta
al nodo anterior de la
lista.

Listas Enlazadas Dobles :

Una Lista Enlazada doble es una estructura de datos lineal que, a diferencia de la lista enlazada simple, permite el acceso a los elementos tanto en orden ascendente como descendente.



Estructura de un nodo :

Cada nodo en una lista enlazada doble contiene tres campos:

- Dato = El valor que almacena en el nodo.
- Puntero al siguiente nodo: Un puntero que apunta al siguiente nodo en la lista.
- Puntero al nodo anterior: Un puntero que apunta al nodo anterior de la lista.

Toil Pointers (Punteros a cola)

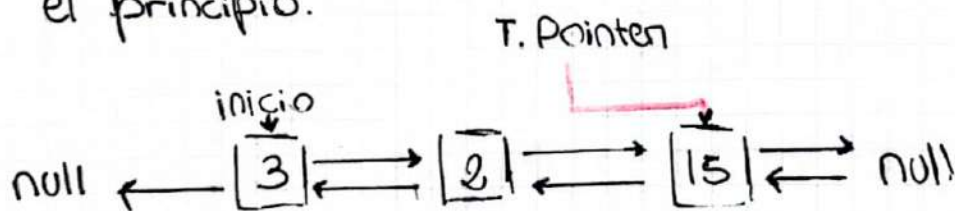
¿Que es un Toil Pointer?

En una Lista Enlazada doble, cada nodo típicamente tiene dos punteros = uno que apunta al nodo siguiente (next) y otro que apunta al nodo anterior (prev). * Un toil pointer es simplemente un puntero que apunta directamente al último nodo de la lista.

• Toil Pointer = Característica

• Acceso rapido al final de la lista:

Si necesitas agregar o eliminar un nodo al final de la lista, tener un toil pointer te permite acceder directamente al último nodo sin tener que recorrer toda la lista desde el principio.



Tiempos de ejecución = Listas Enlazadas Dobles.

• insertar (Aleatorio) : $O(n)$ insertar un elemento en una posición aleatoria requiere recorrer la lista en promedio hasta la mitad, lo que da una complejidad lineal.

• insertar (Front) $O(1)$: Insertar al principio de la lista es una operación constante, ya que solo se necesita actualizar el puntero al inicio.

• insertar (Back) $O(1)$: Similar al anterior, insertar al final también es constante, se necesita actualizar el puntero al final.

• Eliminar (Aleatorio) $O(n)$: Eliminar un elemento aleatorio también requiere recorrer la lista en promedio hasta la mitad.

• Eliminar (Front) $O(1)$: Eliminar el primer elemento es constante.

• Eliminar (Back) $O(1)$: Eliminar el último elemento es constante.

- Buscar (sorted) $O(n)$: Buscar un elemento en una lista ordenada tiene una complejidad lineal en el peor caso.
- Buscar (Unsorted) $O(n)$: Buscar en una lista no ordenada también es lineal.

Inserciones/ Eliminaciones en los extremos, son operaciones eficientes (constantes) ya que mantienen punteros al inicio y al final de la lista.

Operaciones Aleatorias, (posiciones), son menos eficientes (lineales) ya que requieren recorrer la lista para encontrar la posición deseada.

Busqueda, tanto en listas ordenadas como no ordenadas, la búsqueda es lineal.

Tiempos de Ejecución

rtape
brand

	Estructura de Datos	Acceso Aleatorio	Búsqueda	Inserción	Eliminación
A)	Matriz Fija	$O(1)$	$O(n)$	Depende *	Depende *
B)	Matriz Circular	$O(1)$	$O(n)$	Depende *	Depende *
C)	Matriz Dinamica	Amortiguado $O(1)$	$O(n)$	Amortiguado $O(1)$	Amortiguado $O(1)$
D)	Lista Enlazada Simple	$O(n)$	$O(n)$	$O(1)$ al inicio, $O(n)$ en el medio	$O(n)$
E)	Lista Enlazada Doble	$O(n)$	$O(n)$	$O(1)$ al inicio / final, $O(n)$ en el medio	$O(1)$ al inicio / final, $O(n)$ en el medio

Cuestionario = Listas Enlazadas

1. ¿Cual o cuales de las siguientes puntos son ciertos sobre las listas enlazadas en comparación con las matrices?

- A. Es fácil insertar y eliminar elementos en una Lista Enlazada.
- B. El acceso aleatorio no está permitido en una implementación típica de Lista Enlazada.
- C. El tamaño de un arreglo tiene que ser pre-decuido, las listas enlazadas pueden cambiar su tamaño en cualquier momento.
- D. La búsqueda es más fácil cuando una matriz está ordenada, pero no es así con una Lista Enlazada.
- E. Todo lo anterior.

Respuestas =

A. Verdadero. Una de las principales ventajas de las Listas Enlazadas es la facilidad con la que se pueden insertar y eliminar elementos, especialmente al principio o al final de la lista.

B. Verdadero. Para acceder a un elemento en una lista enlazada, es necesario recorrer la lista desde el principio hasta llegar al elemento deseado. (no es eficiente como en una matriz).

C. Verdadero. Las matrices tienen un tamaño fijo al momento de su creación, mientras que las listas enlazadas pueden crecer o decrecer dinámicamente a medida que se añaden o eliminan elementos.

D. Verdadero. En una matriz ordenada, se pueden utilizar algoritmos de búsqueda eficientes como la búsqueda binaria.

En una lista enlazada, incluso si está ordenada, la búsqueda siempre requiere un recorrido secuencial desde el principio.

Característica	L. Enlazada	Matrices
1. Inserción eliminación.	Fácil, especialmente al principio o al fin	Puede ser costoso si se requiere reubicar elementos.
2. Acceso Aleatorio	ineficiente	Efficiente
3. Tamaño	Dinámico	Fijo
4. Búsqueda en datos ordenados	ineficiente	Efficiente (Búsqueda Binaria)

Cuando uson cada una =

- Listos Enlazados = ideales cuando se necesitan frecuentes inserciones y eliminaciones en cualquier posición, pero el acceso aleatorio no es eficiente.
- Matrices = ideales cuando se necesita acceso aleatorio frecuente y el tamaño de la estructura se conoce de antemano.

2) Tenemos un listo doblemente enlazado:

$1 \leftarrow \dots \rightarrow 2 \leftarrow \dots \rightarrow 4 \leftarrow \dots \rightarrow 3$. Si insertamos un 5 al Final, y luego quitamos el 2 de la lista.
¿Cómo será la versión Final?

Al insertar un 5 al Final =

$1 \leftarrow \dots \rightarrow 2 \leftarrow \dots \rightarrow 4 \leftarrow \dots \rightarrow 3 \leftarrow \dots \rightarrow 5$

Al eliminar el "2", debemos ajustar los punteros de los "nodos adyacentes" (1 y 4) para que apunten entre sí.

$1 \leftarrow \dots \rightarrow 4 \leftarrow \dots \rightarrow 3 \leftarrow \dots \rightarrow 5$

nodo adyacente = Un nodo adyacente es aquel que está directamente conectado a otro nodo.

En un listo enlazado simple, un nodo adyacente es aquel nodo que apunta el puntero (siguiente).

En un lista doblemente enlazado, un nodo es adyacente por que su puntero apunta al nodo anterior & al siguiente.

Caracteristicas	nodo adyacente	nodo vecino
- Conexión	Directo	Directo & indirecta.
- Alcance	Un paso	múltiples pasos
- Uso	más específicos en estructuras de datos (Listas Enlazadas)	más general en teorías de Gráficas.

* Gráfico Comparativo (n. adyacente y n. vecino).

3) ¿Por qué nos ayuda un indicador de cola? (Tail Pointer)

1. Proporciona una manera de buscar más rápido.
2. Nos permite insertar y eliminar de la parte posterior en $O(1)$ tiempo Constante.
3. no ayuda en absoluto.

Respuestas:

1. incorrecto. Un indicador de cola, por si solo, no mejora la velocidad de búsqueda en listas enlazadas.

La búsqueda en estas estructuras suele ser lineal, ya que se recorre lo lista nodo por nodo hasta encontrar el elemento deseado. El indicador de cola, apunta al último elemento, pero no proporciona una forma directa de acceder a elementos en posiciones intermedias.

2. Correcto (inserción). Un indicador de cola optimiza significativamente la inserción de elementos al final de una lista enlazada. Al apuntar al último nodo, se puede agregar un nuevo nodo en tiempo constante. $O(1)$.

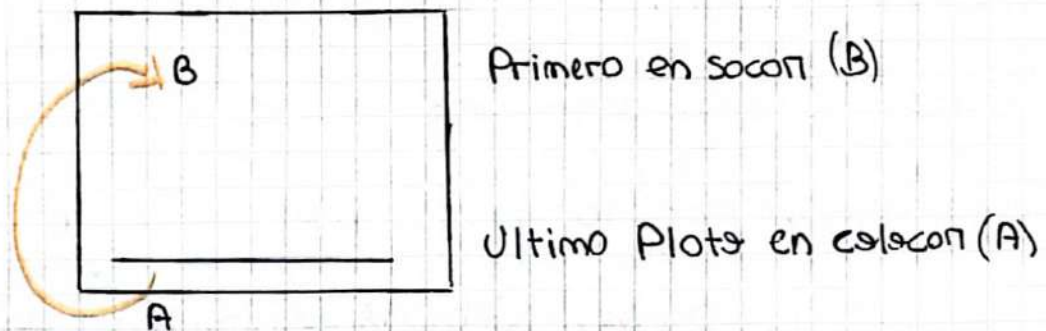
- incorrecto para eliminaciones = Para eliminar el último elemento, se debe recorrer la lista hasta encontrar el penúltimo nodo y actualizar su puntero.

3. incorrecto. Optimiza la inserción al final de la lista

Stack & Queues =

¿Qué es un Stack (Pila)?

Un Stack es una estructura de datos lineal que sigue el principio LIFO (Last in, First out), el último elemento en entrar es el primero en salir.



características = El acceso a los elementos se realiza únicamente por un extremo llamado "Top".

Orden LIFO

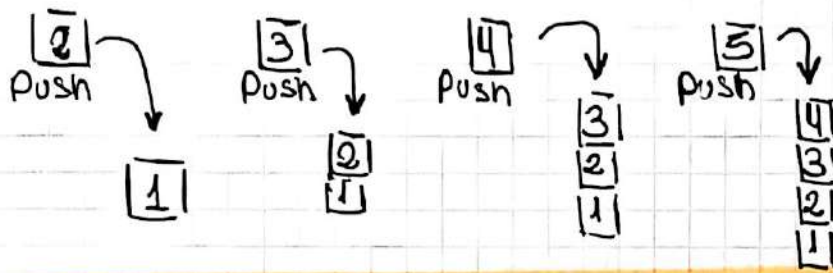
Operaciones Básicas = A) Push = Agregar un elemento al ^{pila.} tope de la pila.
B) Pop = Eliminar el elemento del tope de la pila.

C) Peek = Observar el elemento del tope sin eliminarlo.

F) Deque = Para eliminar un elemento de una cola

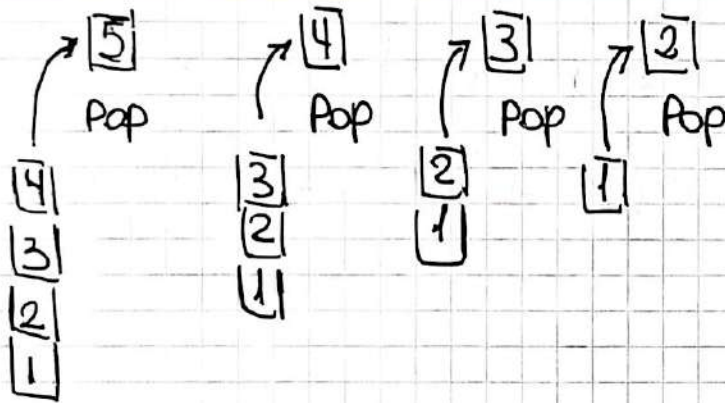
D) IsEmpty = Verificar si la pila está vacía

G) FIFO = Primero en entrar, primero en salir. el Enqueue = Insertar un elemento en una cola.



Tope = El Tope de una Pila (Stack) es el Final.

* El ultimo Plato que coloco es el que quedo arriba, el que llamamos Tope.



* Implementando un Stack con arrays =

Un Array es una estructura de datos de tamaño fijo, por lo que al implementar un Stack con un array, debemos considerar un limite máximo de elementos.

- Se utiliza un indice para apuntar al "tope" de la pila,
- El primer elemento del array se considera la "base" de la Pila.

Lo base de una Pila en un array es, el Primer elemento que se inserto.

implementando un stack con Listos Enlazados

Una Lista Enlazada es una estructura de datos dinamica, lo que significa que su tamaño puede crecer o decrecer según sea necesario.

Representación =

- Cada nodo de la lista contiene un dato y un puntero al siguiente nodo.
- El "tope" de la pila apunta al primer nodo de la lista.

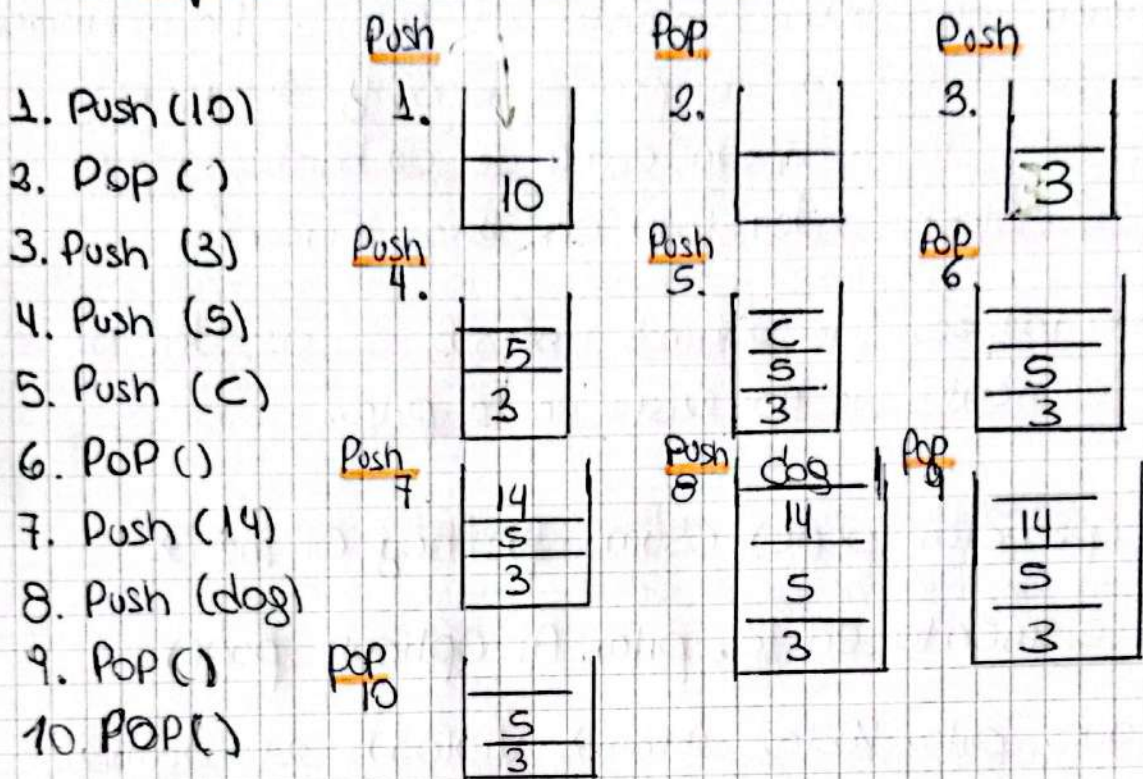
Stack (Pila) Implementando en Array y Lista Enlazada

Operación	Array	Lista Enlazada
<u>Push</u>	Incrementa el índice del tope. Asigna el nuevo elemento a la posición del tope.	Crea un nuevo nodo. Asigna el valor del nuevo elemento al nodo. * Hace que el nuevo nodo apunte al nodo anterior (tope). Actualiza el tope para que apunte al nuevo nodo.
<u>Pop</u>	Devuelve el valor del elemento en el tope. Decrementa el índice del tope.	Devuelve el valor del elemento en el tope. Actualiza el tope para que apunte al siguiente nodo. Libera la memoria del nodo eliminado.
<u>Peek</u>	Devuelve el valor del elemento en el tope. No modifica la pila.	Devuelve el valor del elemento en el tope. No modifica la pila.
<u>IsEmpty</u>	Compara el índice del tope con un valor que indica pila vacía (generalmente -1).	Compara el tope con null (o cualquier valor que indique el final de la lista).

Customer Service:
Web: www.intertapepolymer.com
Email: info@intertapepoly.com
800-IPG-8273 / 800-474-8273

intertape polymer group

Ejemplo = Push y Pop (Stack)



Out+Put = (2) , (5) , (6) , (9) , (10)
 10 C S 14 5

⊛ ¿Por qué se muestran los resultados del pop en lo Solido?

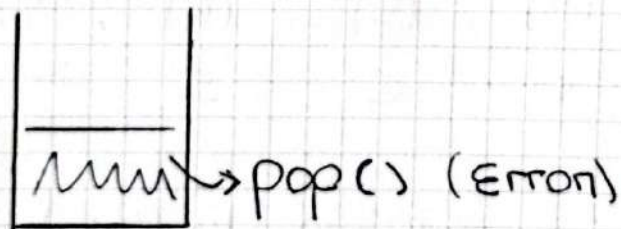
• Seguimiento del proceso = Al mostrar los "resultados" del pop, podemos ver claramente como evoluciona la pila a medida que se realizan las operaciones. Es útil para entender el funcionamiento de la Pila y depurar posibles errores.

¿Que pasa si aplicamos $\text{pop}()$ a un Stack Vacío?

- Error de desbordamiento de pila = La mayoría de los lenguajes de programación y estructuras de datos lanzarán un "error de desbordamiento de pila" (Stack underflow) en esta situación.

Esto indica que estamos tratando de acceder a un elemento que no existe en la pila.

- La operación $\text{pop}()$ asume que hay al menos un elemento en la pila. Al aplicar $\text{pop}()$ a una pila vacía, estamos violando esta premisa básica.



Queues =

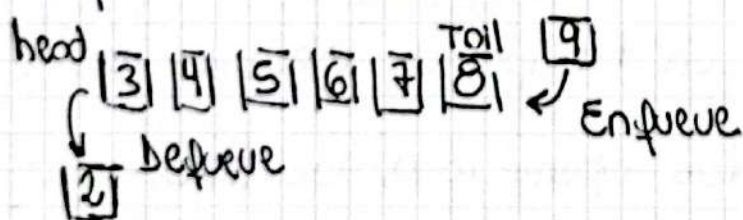
Una 'cola, (Queues)' es una estructura de datos lineal que sigue el principio 'FIFO'; el primer elemento en entrar es el primero en salir.

¿Cómo funciona?

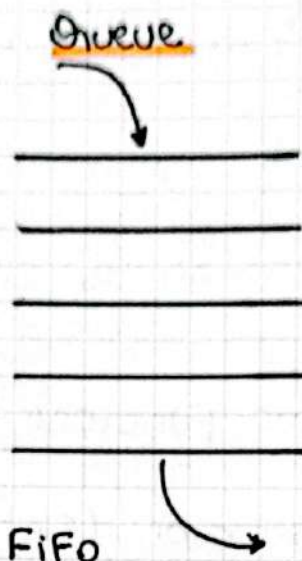
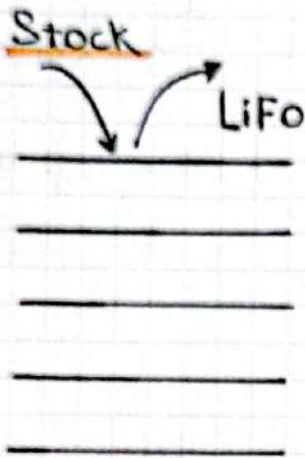
• insersion (Enqueue) = Los nuevos elementos se añaden al final de la cola.

• Eliminación (Dequeue) = Los elementos se eliminan del principio de la cola.

Representación Gráfica:



Diferencia (Stack y Queue)



* LIFO = Último elemento en entrar,
primero en salir.

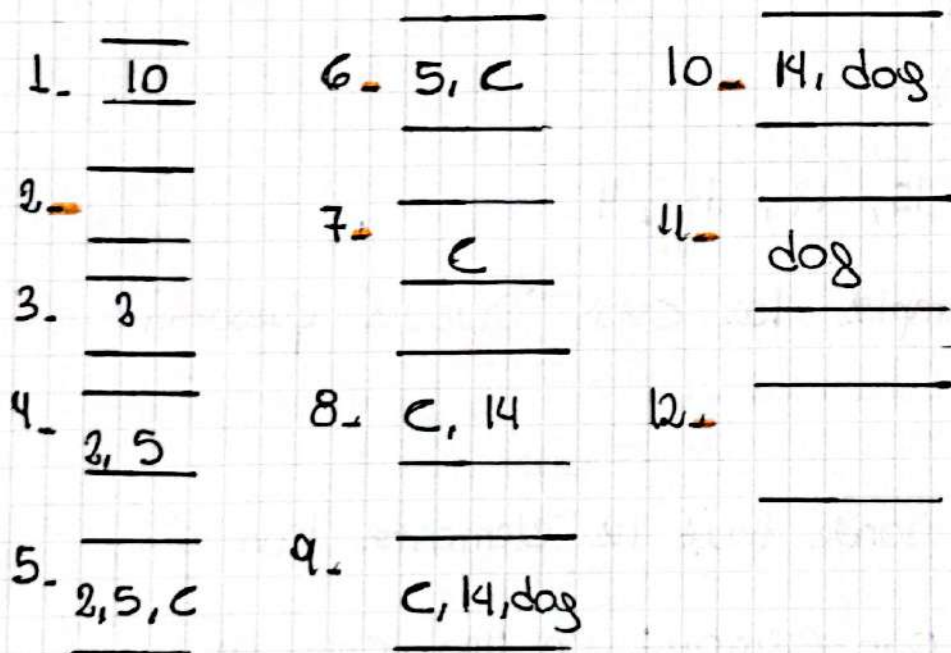
* FIFO = Primer elemento en
entrar, es el primero
en salir

tiempos de ejecución (Queues, Stacks)

Podemos obtener un tiempo de ejecución de $O(1)$ en todas las operaciones de pilas y colas. Esto se debe a que solo accedemos a los "extremos".

A) Ejemplo = Queue (Utilizando Push y Pop)

- | | |
|-------------|--------------|
| 1. Push(10) | 7. Pop() |
| 2. Pop() | 8. Push(14) |
| 3. Push(2) | 9. Push(dog) |
| 4. Push(5) | 10. Pop() |
| 5. Push(c) | 11. Pop() |
| 6. Pop() | 12. Pop() |



Output = (2), (6), (7), (10), (11), (12)
 10 2 5 c 14 dog

B) Ejemplo = Queue (utilizando Enqueue, Dequeue)

1. Enqueue (10)

2. Enqueue (15)

3. Enqueue (17)

4. Dequeue ()

5. Dequeue ()

6. Enqueue (14)

7. Dequeue ()

8. Dequeue ()

9. Enqueue (4)

10. Dequeue ()

10	15	17	14	4
---------------	---------------	---------------	---------------	--------------

Output = 10, 15, 17, 14, 4

En ambos ejemplos las colas "Queue" quedaron vacías,

Situaciones donde todos los elementos han sido procesados, por ejemplo, en una cola de tareas, una vez que todas las operaciones han sido completadas, la cola (queue) queda vacía.

Quiz = Stack y Queues

A) Tenemos una pila, (stack) (con la parte

Superior a la izquierda, (topo)) de: $[-, -, 5, 4, 3, 2]$

¿Qué aspecto tendrá la pila Stack?

1. Pop()

2. Push(5)

3. Pop()

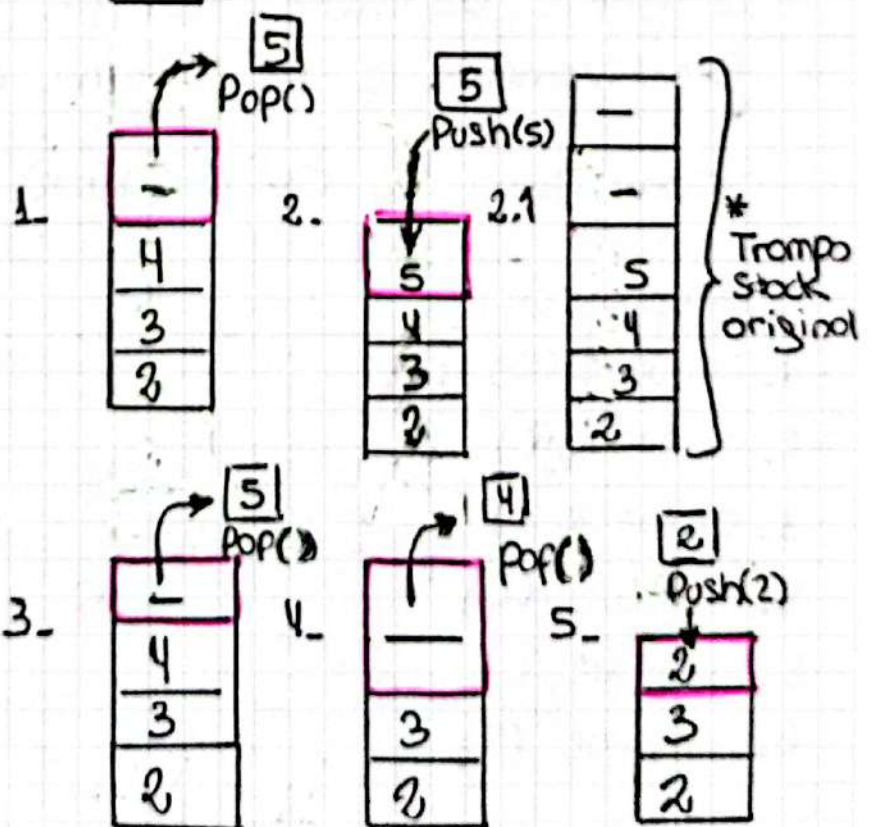
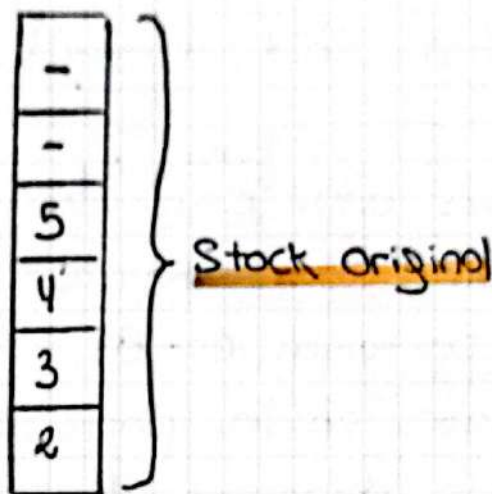
4. Pop()

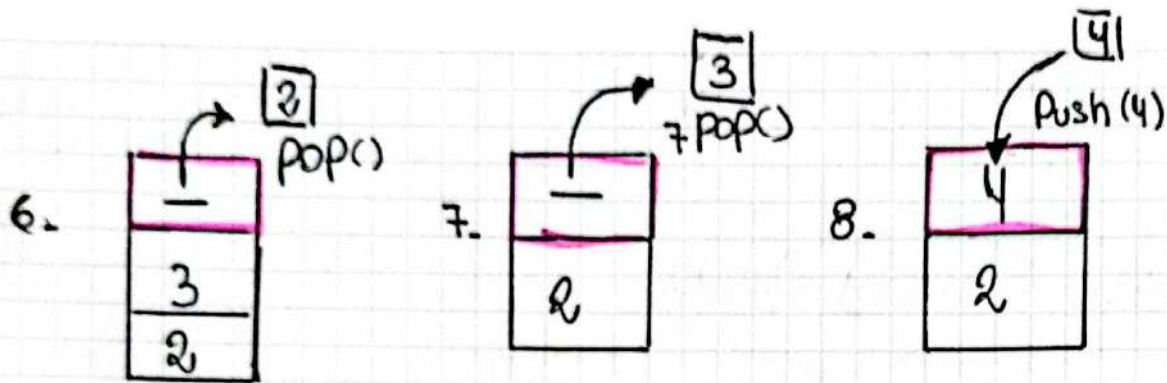
5. Push(2)

6. Pop()

7. Pop()

8. Push(4)





Resultado final = $[-, -, -, -, 4, 2]$

*En el tercer punto comienza a contarse cuantos

pop() hay, en total son 4.

B) tenemos una Queue "cola" que utiliza una matriz circular (con el Frente actualmente en 5, y la parte posterior en 8).

Se inserta en la parte posterior, y elimina en la parte delantera. Así que la matriz se moverá efectivamente a la derecha a medida que avanza.

¿Cómo sero la cola (Queue) de $[-, -, 5, 7, 8, -, -]$

despues de:

1. enqueue (5)
2. dequeue ()
3. dequeue ()
4. enqueue (0)
5. dequeue (),
6. enqueue (4)

Queue

- Cola inicial = $[-, -, 5, 7, 8, -, -]$

1. Enqueue(5) = Se inserta al Final, en el espacio vacío.

. Cola resultante = $[-, -, 5, 7, 8, 5, -]$

2. Dequeue() = Se elimina el elemento del Frente ⁽⁻⁾ (Vacio)

. Cola resultante = $[-, -, 5, 7, 8, 5, -]$

* El elemento eliminado es reemplazado por un -.

3. Dequeue() = Se elimina el elemento del Frente (-).

. Cola resultante = $[-, -, 5, 7, 8, 5, -]$

* El Elemento eliminado es reemplazado por un -.

4. Enqueue(Q) = Se inserta al Final en el espacio vacío la "Q".

. Cola resultante = $[-, -, 5, 7, 8, 5, Q]$

5. Dequeue() = Se elimina el elemento del Frente (-).

. Cola resultante = $[-, -, 5, 7, 8, 5, Q]$

6. Enqueue(4) = Se inserta 4 al Final, pero como es matriz circular fue a parar al índice 0.

. Cola resultante = $[4, -, 5, 7, 8, 5, Q]$

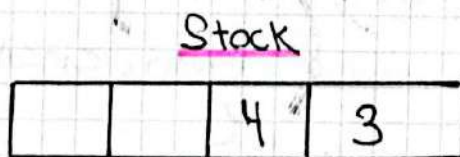
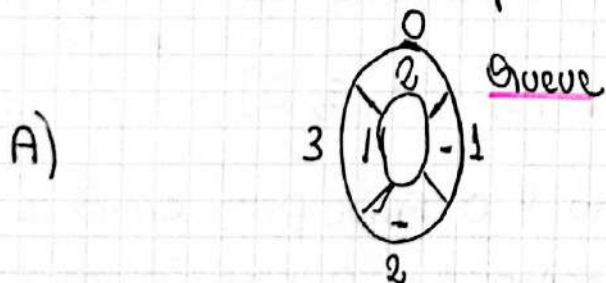
En una Queue implementado en un array, el Frente y la parte trasera, "envuelven" el array cuando llegan al Final. Esto significa que aunque la cola se mueve efectivamente hacia la derecha, los elementos no se desplazan físicamente en el array. En su lugar, los índices de Frente y trasera se actualizan para reflejar las nuevas posiciones.

c) Tenemos una Pila (Stack) y una cola (Queue)

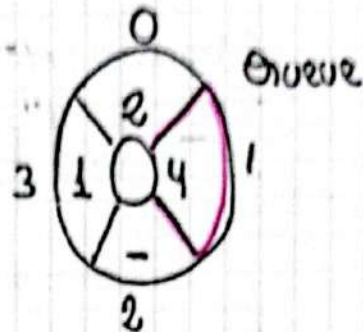
$[-, -, 4, 3]$ y $[-, -, 2, 1]$ respectivamente.

Cuando se sale de la pila (Stack), se pone en cola, (enfrente) en la Queue, implementado, como una matriz circular.

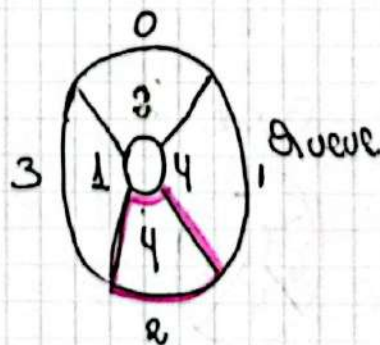
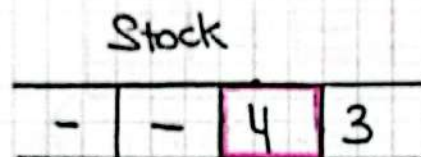
(La parte delantera de la cola está actualmente en 2 (Queue) y la parte trasera en 1 (Queue).



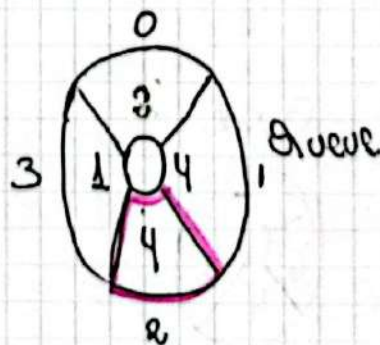
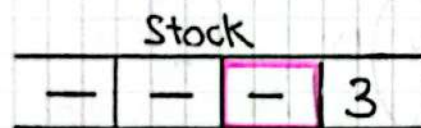
a) pop()



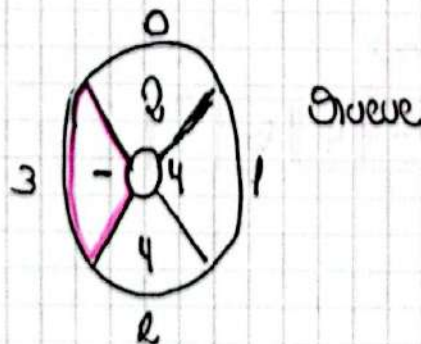
b) push(4)



c) pop()



d) dequeue()



e) push(5)

Stack



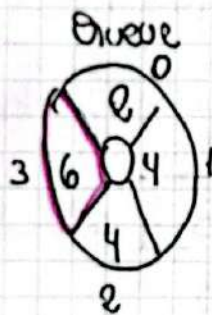
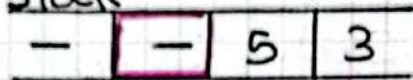
f) push(6)

Stack



g) pop()

Stack



h) dequeue

Queue



i) push(4)

Stack

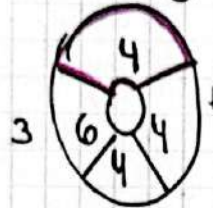


j) pop()

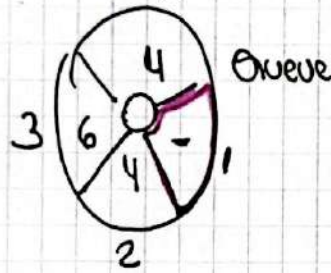
Stock

-	-	5	3
---	---	---	---

Queue



k) enqueue()



tenemos una pila, y una Cola, ambas inicialmente con algunos elementos. Lo particular es que cuando sacamos un elemento de la pila (Stack) (pop), lo agregamos al principio de la Cola (enqueue). Al ser circular se agrega del principio y se elimina por el final.

Resultado = Stock = [-, -, 5, 3]
Queue = [4, -, 4, 6]

Tiempos de ejecución

- bubble Sort
- Selection Sort
- insertion Sort
- Quick Sort
- merge Sort

Complejidad Algorítmica

• $\Omega(n)$ = Representa el mejor caso posible para un algoritmo.

• $\Theta(n)$ = Representa el promedio de un algoritmo.

• $O(n)$ = Representa el peor escenario de un algoritmo.

Algorithms)

una lista de elementos (números, palabras, objetos) en un orden específico.

Este orden puede ser ascendente (menor a mayor) o descendente (de mayor a menor).

Bubble Sort = Compara elementos adyacentes e intercambia sus posiciones si están en el orden incorrecto.

tiempos de ejecución =

mejor tiempo = $\Omega(n)$; indica el mejor de los casos, que el algoritmo recorra el array (matriz) una sola vez. Esto ocurre cuando los datos ya están ordenados y el algoritmo verifica que estén en el orden correcto.

"n (omega)".

Sorting Algorithms

Los Algoritmos de Ordenamiento (Sorting Algorithms) son una serie de instrucciones que se utilizan para organizar una lista de elementos (números, palabras, objetos) en un orden específico.

Este orden puede ser ascendente (menor a mayor) o descendente (de mayor a menor).

Bubble Sort = Compara elementos adyacentes e intercambia sus posiciones si están en el orden incorrecto.

Tiempo de ejecución =

Mejor tiempo = $\Omega(n)$; indica, en el mejor de los casos, que el algoritmo recorrerá el array (matriz) una sola vez. Esto ocurre cuando los datos ya están ordenados y el algoritmo verifica que estén en el orden correcto.
"n (omega)".

Ejemplo: Ordenar la matriz $[-2, 45, 0, 11, -9]$:

-2	45	0	11	-9
----	----	---	----	----



-2	45	0	11	-9
----	----	---	----	----



-2	0	45	11	-9
----	---	----	----	----



-2	0	11	45	-9
----	---	----	----	----

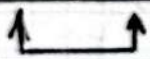


-2	0	11	-9	45
----	---	----	----	----

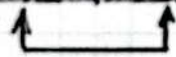
-2	0	11	-9	45
----	---	----	----	----



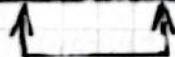
-2	0	11	-9	45
----	---	----	----	----



-2	0	11	-9	45
----	---	----	----	----



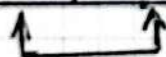
-2	0	-9	11	45
----	---	----	----	----



-2	0	-9	11	45
----	---	----	----	----



-2	0	-9	11	45
----	---	----	----	----



-2	-9	0	11	45
----	----	---	----	----



12 Resultado

$[-2, -9, 0, 11, 45]$

Resultado

Selection Sort

El "Selection Sort" es un algoritmo de "ordenamiento", funciona dividiendo la lista en dos partes; una ordenada y otra desordenada.

En cada iteración, se busca el elemento más pequeño (o más grande, dependiendo del orden deseado) en la parte desordenada y se coloca al principio de la parte ordenada.

Ejemplo = [64, 25, 12, 22, 11]

1. Primera iteración =

. El elemento más pequeño es 11.

. Se inter-cambia con 64.

. Lista = [11, 25, 12, 22, 64]

* Todo lo que está a la izquierda está ordenado, y lo que está a la derecha sin ordenar

* [8, 10, 2, 3], [2, 10, 8, 3]

- (1) -

- (1) -

iteración =

ordenando los elementos desde el índice hasta el final, el más pequeño es

inter-cambia con 25.

[11, 12, 25, 22, 64]

momento.

Selection Sort

El "Selection Sort" es un algoritmo de "ordenamiento", funciona, dividiendo la lista, en dos partes; una, ordenada, y otra, desordenada.

En cada iteración, se busca el elemento más pequeño (o más grande, dependiendo del orden deseado) en la parte desordenada y se coloca al principio de la parte ordenada.

Ejemplo = [64, 25, 12, 22, 11]

- 1. Primera iteración =

. El elemento más pequeño es 11.

. Se inter-cambia, con 64.

. Lista = [11, 25, 12, 22, 64]

- 2. Segunda iteración =

. Considerando los elementos desde el índice 1 (25) hasta el final, el más pequeño es 12

. Se inter-cambia con 25.

. Lista = [11, 12, 25, 22, 64]

- 3. Así sucesivamente.

cuestionario = Bubble Sort y Selection Sort

1.- ¿Qué aspecto tendrá la lista $\{4, 5, 6, 8, 22, 4, 3\}$ después de 1 pasada utilizando Bubble Sort?

Respuesta = El aspecto después de 1 pasada utilizando Bubble Sort es = $\{1, 5, 6, 8, 4, 3, 22\}$

2.- ¿Qué aspecto tendrá la lista $\{10, 5, 7, 14, 25, 6\}$ después de 1 pasada utilizando bubble Sort?

Respuesta = El aspecto después de 1 pasada utilizando Bubble Sort es = $\{5, 7, 10, 14, 6, 25\}$

3.- ¿Qué aspecto tendrá la lista $\{10, 5, 7, 14, 25, 6\}$ después de una pasada, utilizando Selection Sort?

Respuesta = El aspecto después de 1 pasada, utilizando Selection Sort es = $\{5, 10, 7, 14, 25, 6\}$

4.- ¿Qué aspecto tendrá la lista $\{10, 5, 7, 14, 25, 6\}$ después de 2 pasadas utilizando Selection Sort?

Respuesta = El aspecto después de 2 pasadas utilizando Selection Sort es = $\{5, 6, 7, 14, 25, 10\}$

insertion Sort

El insertion Sort es un Algoritmo de Ordenamiento que construye una secuencia ordenada, insertando cada elemento en su posición correcta dentro de la subsecuencia ordenada ya construida.

En lugar de encontrar el mínimo en la parte sin ordenar, toma el número más cercano a la parte sin ordenar y lo inserta en la parte ordenada.

Si todo el array está ordenado, entonces hace una sola pasada y devuelve el array ordenado.

Ejemplo = $\{29, 10, 14, 37, 14\}$

$\{29, \downarrow, 14, 37, 14\}$
 $\uparrow$
 10

$\{10, 29, 14, 37, 14\}$

$\{10, 29, \downarrow, 37, 14\}$
 $\uparrow$
 14

$\{10, 14, 29, 37, 14\}$

$\{10, 14, 29, 37, \downarrow\}$
 $\uparrow$
 14

$\{10, 14, 14, 29, 37\}$

Recursión:

La Recursión es una técnica que permite a una función llamarse a sí misma.

- Una Función se llama a sí misma = Lo clave de la recursión es que una función puede invocarse a sí misma, pero con diferentes parámetros o en un contexto modificado.
- Caso Base = Para evitar un bucle infinito, toda función recursiva debe tener un "Caso base". Este es un conjunto de condiciones que, al cumplirse, detiene la recursión y devuelve un valor.
- Caso recursivo = Es la parte de la función donde se realiza lo llamado recursividad.

Quick Sort:

Quick Sort (o ordenamiento rápido) es un Algoritmo de ordenamiento muy eficiente basado en la técnica de 'divide y vencerás'. Es capaz de ordenar en $\log(n)$.

1. Elección de un pivote = Se selecciona un elemento del conjunto de datos, al que llamaremos pivote.
2. Particionamiento = Se reordenan los elementos del conjunto de datos de tal forma que todos los elementos menores al "pivote" queden a su izquierda y los mayores a su derecha.

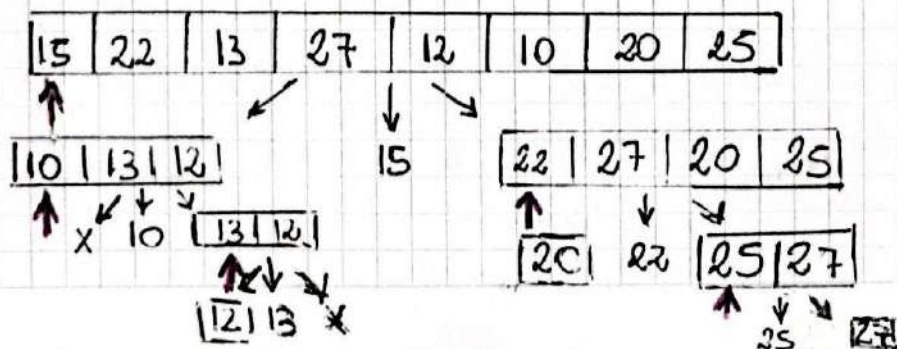
3. Llamadas recursivas: Se aplica el algoritmo de forma recursiva a los dos sublistas resultantes de la partición.

Características:

- Velocidad: En promedio, es uno de los algoritmos de ordenamiento más rápidos.
- Recursividad: Se define en términos de sí mismo, lo que permite dividir el problema en sub-problemas más pequeños.
- Inestabilidad: Puede alterar el orden relativo de elementos iguales.
- Sensibilidad a la elección del pivote: La eficiencia del algoritmo depende en gran medida de cómo se seleccione el pivote.

* Ejemplo = tenemos una caja de bombones de diferentes sabores y los ordenamos por intensidad de sabor (del más suave al más intenso).

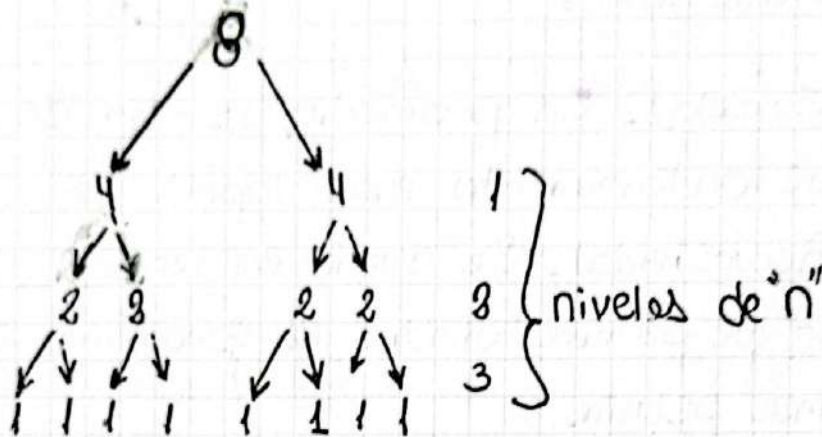
* Ejemplo 2 = tenemos un array $\{15, 22, 13, 27, 12, 10, 20, 25\}$
Queremos ordenarlo utilizando Quick Sort.



$\left\{ \begin{array}{|c|c|c|c|c|c|c|} \hline 10 & 12 & 13 & 15 & 20 & 22 & 25 & 27 \\ \hline \end{array} \right\}$
↗ Pivote inicial.
Resultado Final

* Siempre Seleccionamos el "pivote" del lado de la izquierda para luego sub-dividir la matriz.

* Ejemplo (anterior)



X	n
8	3
16	4
32	5
64	6
128	7

$\log(n)$
Grafico.

Quiz insertion Sort y Quick Sort

1. ¿Qué aspecto tendrá la lista $\{4, 10, 5, 17, 3, 8, 11\}$ después de 2 pasadas (es decir, 2 movimientos) utilizando Insertion Sort. (Supongamos que 4 no cuenta como pasada).

- [4] (El primer elemento ya está ordenado).
- [4, 10] (El 10 se inserta al final)
- [4, 5, 10] (El 5 se inserta antes que el 10)
- [4, 5, 10, 17] (El 17 se inserta al final)
- [3, 4, 5, 10, 17] (El 3 se inserta al principio)
- [3, 4, 5, 8, 10, 17] (El 8 se inserta antes del 10)
- [3, 4, 5, 8, 10, 11, 17] (El 11 se inserta después del 10)
- Respuesta = - [4, 5, 10]

* El ordenamiento por inserción es un algoritmo bastante intuitivo.

¿Cómo funciona?

• al inicio = consideramos el primer elemento como un subconjunto ordenado.

* b iteración: Para cada elemento, restante:

- Comparación: Se compara el elemento actual con los elementos del subconjunto ordenado.
- Desplazamiento: Si el elemento actual es menor, se desplazan los elementos mayores hacia la derecha para hacer espacio.
- Inserción: Se inserta el elemento actual en la posición correcta.

- 2 ¿Que puede hacer que "Quick Sort" se convierta en un algoritmo n^2 ?

- a: Si la longitud de la lista de entrada es demasiado larga.
- b: Una elección continuo de malos puntos de pivote. (Todos menores o mayores que).
- c: Si la lista ya está ordenada.
- a: Esto tampoco es correcto. La longitud de la lista puede influir en el tiempo de ejecución total, pero no la causa directa de que Quick Sort degenera $O(n^2)$.

- b. Una elección continua de malos puntos de pivote: Si el pivote elegido en cada partición es siempre el elemento más pequeño o más grande, las particiones serán muy desbalanceadas, llevando a una recursión desbalanceada, y, por lo tanto, a una complejidad $O(n^2)$. (Esto es correcto)
- c. Si la lista ya está ordenada: Si la lista está ordenada o casi ordenada, la elección de un pivote al azar (como el primero o último elemento) tenderá a generar particiones desbalanceadas, lo que conduce $O(n^2)$. (Esto es correcto)

Merge Sort = "Merge Sort" es un algoritmo de ordenación en donde, tiene una forma de dividir los datos como en Quick Sort, excepto que no requiere seleccionar un pivote para obtener los tiempos de ejecución " $n \log n$ ".

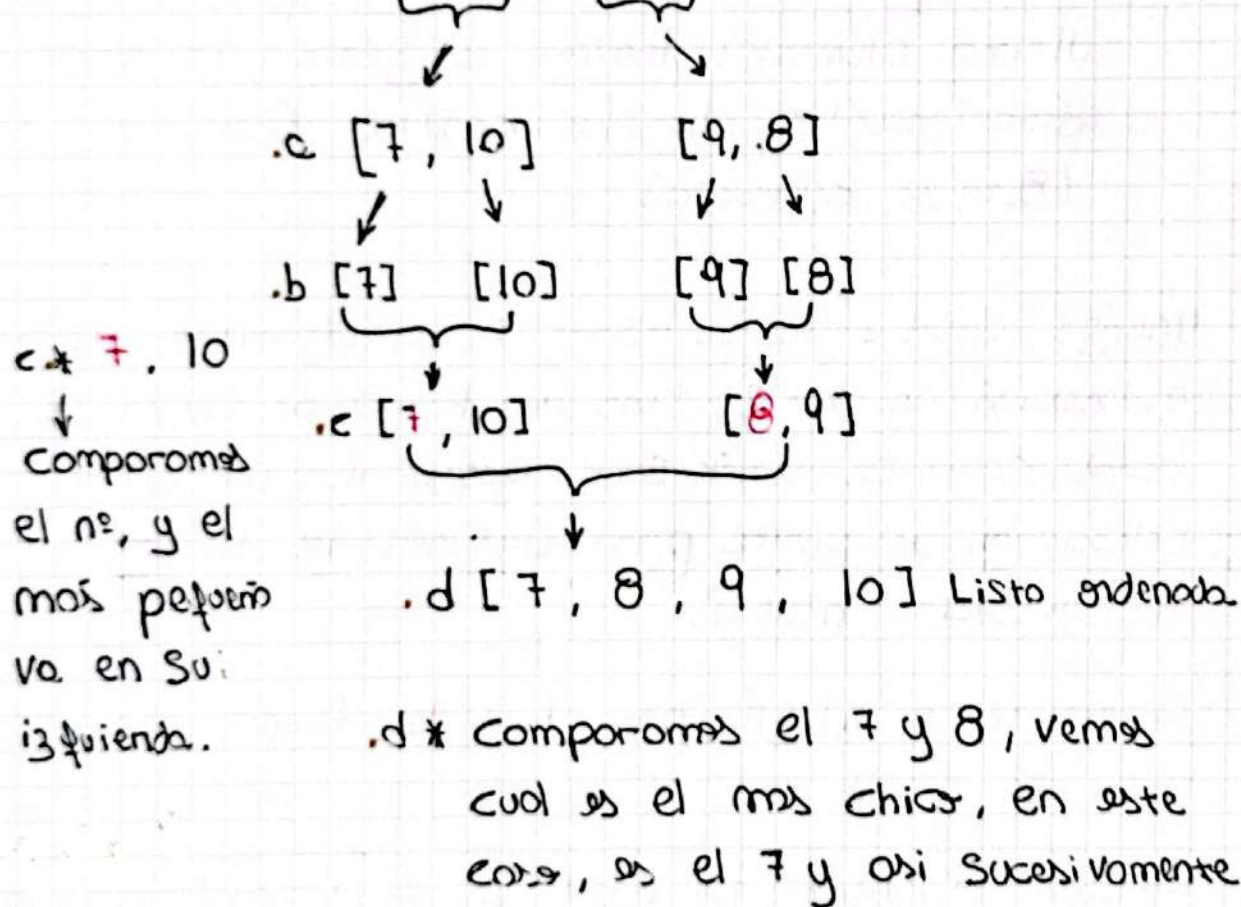
Se ejecuta a " $n \log n$ " para todos los tiempos de ejecución.

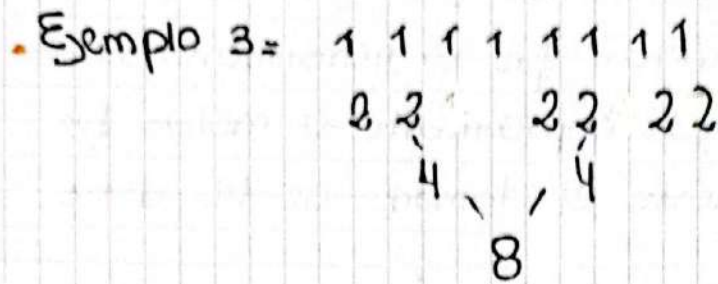
Funciona dividiendo los datos en trozos más pequeños. Luego los recombina en un algoritmo ordenado.

Ejemplo: $[3, 7, 2, 8, 5, 1, 9, 6]$

- Se divide en $= [3, 7, 2, 8]$ y $[5, 1, 9, 6]$
- Se divide hasta obtener $= [3], [7], [2], [8], [5], [1], [9], [6]$
- Se combinan de a dos $= [3, 7], [2, 8], [5, 1], [9, 6]$
- Se combinan nuevamente $= [2, 3, 7, 8], [1, 5, 6, 9]$
- Finalmente, se combinan las dos últimas listas para obtener la lista ordenada $= [1, 2, 3, 5, 6, 7, 8, 9]$

Ejemplo 2: $[7, 10, 9, 8]$

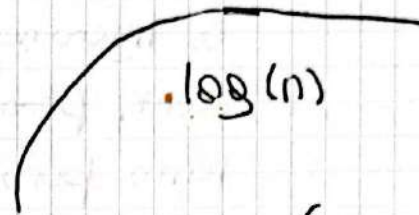




* Si tenemos 8 datos tenemos $n = 3$.

* 4 2
8 3
16 4
32 5
64 6
128 7
...
datos n

* Grafico =



$\log(n)$

* Vamos a tener un tiempo $\begin{cases} n \cdot \log n \\ O(n \cdot \log n) \\ O(n \cdot \log n) \end{cases}$

Tiempo de Ejecucion = [Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort]

- mejor caso = El tiempo mínimo que tarda el algoritmo en ejecutarse para un conjunto de datos.
- Promedio = El tiempo promedio que tarda un algoritmo en ejecutarse para diferentes conjuntos de datos.
- Peor caso = El tiempo máximo que tarda el algoritmo en ejecutarse para un conjunto de datos.

. Bubble Sort = Su tiempo "promedio" y "peor caso" es cuadrático ($O(n^2)$), lo que significa que su rendimiento se degrada rápidamente a medida que aumenta el tamaño de los datos.

. Selection Sort = También tiene un "tiempo promedio" y "peor caso" cuadrático ($O(n^2)$). Es ligeramente más eficiente que Bubble Sort, pero sigue siendo considerado lento para grandes conjuntos de datos.

. Insertion Sort = Es más eficiente que los anteriores en conjuntos de datos pequeños o parcialmente ordenados. Su "tiempo promedio" es cuadrático ($O(n^2)$), pero su "mejor caso" es lineal ($O(n)$), lo que lo hace más rápido cuando los datos están casi ordenados.

. Quick Sort = Es uno de los algoritmos de ordenamiento más rápidos en promedio. Su tiempo promedio es logarítmico lineal ($O(n \log n)$), lo que significa que escala muy bien con grandes conjuntos de datos. Sin embargo, su peor caso es cuadrático ($O(n^2)$), aunque es poco probable que ocurra en la práctica.

- Merge Sort: también tiene un tiempo promedio logarítmico lineal ($O(n \log n)$), lo que lo hace muy eficiente. A diferencia de Quick Sort, su peor caso es logarítmico lineal ($O(n \log n)$), lo que lo convierte en un algoritmo muy estable y predecible.

Estable y no Estable =

- Algoritmo Estable = Si al ordenar la lista, y los elementos con el mismo valor mantienen su orden original, decimos que el algoritmo es estable. Es decir, si un elemento "A" estaba antes que un elemento "B" con el mismo valor, después de ordenar, "A" seguirá estando antes que "B".
- Algoritmo no estable = Si al ordenar, los elementos con el mismo valor pueden cambiar su orden relativo, el algoritmo es = no estable. En este caso, no hay garantía de que el orden original de los elementos iguales se preserve.

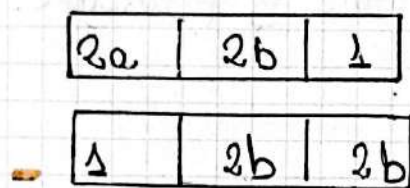
. Algoritmo Estable =

- . Bubble Sort
- . Insertion Sort
- . Merge Sort

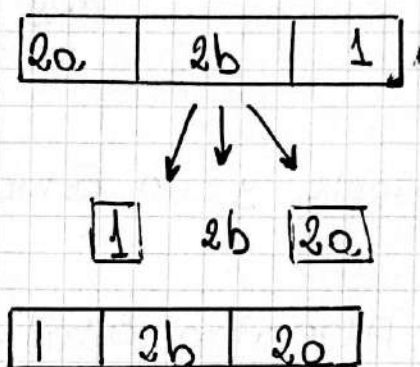
. Algoritmo no Estable =

- . Selection Sort
- . Quick Sort.

. Ejemplo = . Selection Sort



. Quick Sort



. Merge = Merge Sort y Estable

1. ¿Cómo puede Merge Sort ordenar en tiempo $n \log n$?

. utiliza el método de división y conquista

A * Divide recursivamente la lista en dos mitades hasta que cada sublista tenga un solo elemento.

B * Conquista = combina (o fusiona) las sublistas ordenadas de manera eficiente para formar una lista ordenada más grande.

- C* Fusión = compara el primer elemento de cada sublista.

El elemento más pequeño se copia a uno nuevo listo. Se repite el proceso con los elementos restantes de ambas sublistas hasta que se hayan copiado todos los elementos a lo nuevo listo.

- 2. ¿Por qué Merge Sort tiene el tiempo medio de ejecución más rápido? (Run times)

Porque su peor velocidad es $n(\log n)$.

Esto garantiza que pase lo que pase, siempre se ejecutará a $n(\log n)$, dándonos el mejor tiempo medio de ejecución.

- 3. ¿Qué significa estabilidad en un algoritmo de ordenación?

La Estabilidad en un algoritmo de ordenación se refiere a cómo el algoritmo maneja elementos iguales.

Arboles (Trees)

Los árboles son una estructura de datos que se utiliza para representar información de manera jerárquica.

¿Cómo Funciona un Árbol (trees)?

- nodes = Cada elemento dentro del árbol se llama nodo. Un nodo contiene información y puede tener enlaces a otros nodos, llamados hijos.
- Root = Es el nodo superior del árbol, no tiene ningún padre
- hijos = Son los nodos que se encuentran un nivel por debajo de otro nodo.
- Padre = Es el nodo inmediatamente superior a otro nodo.
- Hojas = Son los nodos que no tienen hijos.
- Subárbol = Es cualquier parte del árbol que a su vez es un árbol.

Características =

- Organización de datos = Son ideales para representar jerarquías. Como sistemas de archivos, estructuras de directorios & Clasificaciones.
- Búsquedas eficientes: Los árboles binarios de búsqueda permiten buscar elementos de manera rápida y ordenada.
- Algoritmos de Ordenamiento: Algunos algoritmos de ordenamiento como el Quick Sort, utilizan árboles.
- Compilación = Los compiladores utilizan árboles sintácticos para analizar el código fuente.

Tipos de árboles =

- Árboles Binarios = Cada nodo tiene a lo sumo dos hijos.
- Árboles binarios de búsqueda = Los valores de los nodos se ordenan de forma específica para permitir búsquedas eficientes.
- Árboles AVL = Son árboles binarios auto balanceados, lo que garantiza tiempos de búsqueda, inserción y eliminación eficiente.

• Árboles B: Se utilizan en bases de datos para almacenar índices de gran tamaño.

• Árboles de prefijos: Se utilizan para almacenar y buscar cadenas de caracteres.

Árboles (Trees) Búsqueda Binaria:

¿Qué es un Árbol Binario de Búsqueda?

Un ABB, los nodos contienen valores y se organizan de tal manera que =

• Cada nodo tiene a lo sumo dos hijos = uno a la izquierda y otro derecho.

• El valor de cada nodo es mayor que todos los valores de su subárbol izquierdo.

• El valor de cada nodo es menor que el subárbol derecho.

Características =

A) Búsqueda =

• Se comienza en la raíz.

• Si el valor buscado es igual al valor del nodo actual, se encuentra.

• Si el valor buscado es menor, se continúa buscando en el subárbol izquierdo.

• Si el valor buscado es mayor, se continúa buscando en el subárbol derecho.

B) Inserción =

- Se realiza una búsqueda, como si se quisiera encontrar el elemento.
- Si el elemento no se encuentra, se inserta un nuevo nodo en la posición correspondiente, manteniendo el orden del árbol.

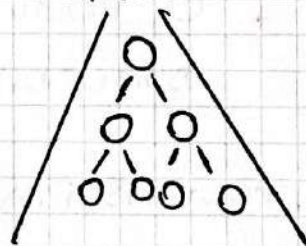
C) Eliminación =

- Se encuentra el nodo a eliminar.
- Si el nodo no tiene hijos, se elimina directamente.
- Si el nodo tiene un hijo, se reemplaza por ese hijo.
- Si el nodo tiene dos hijos, se reemplaza por su sucesor inmediato (el nodo más a la izquierda del subárbol derecho) y luego se elimina el sucesor.

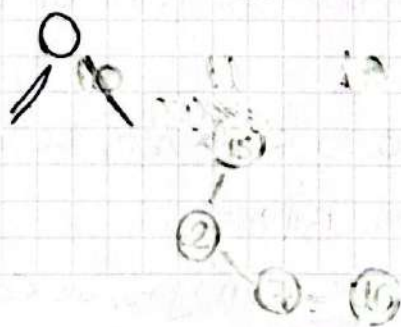
Ejemplo = Binario

Búsqueda

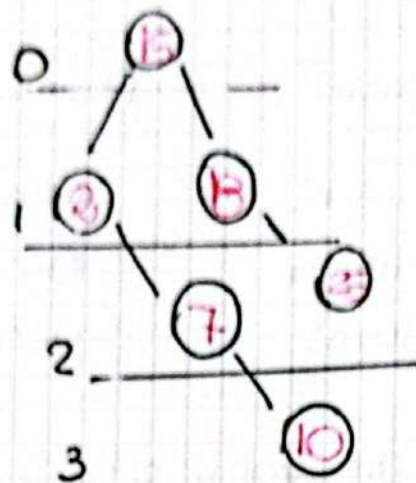
Árbol



menor < X > mayor



15 2 7 10 18 25 ABB

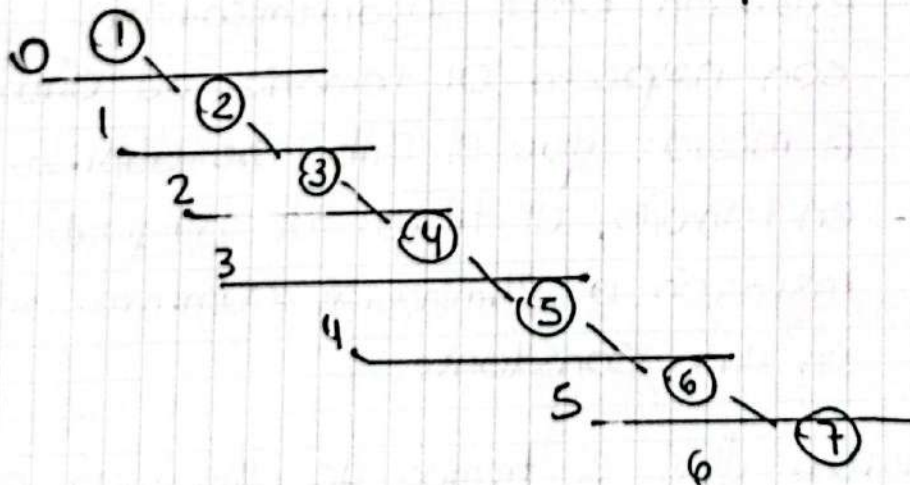


* tiene 3 niveles.

* Si fuéramos encontrar el 7 nos costo 2 niveles por 15 y 2.

* Si fuéramos buscar el 6 (no estar) nos costo 3 niveles (15, 2, 7).

- 1, 2, 3, 4, 5, 6, 7 * El peor Ejemplo (detalle).



Tiempos de ejecución (Busqueda Binaria)

operación	Promedio	Peor caso
- Busqueda	$\Theta(\log n)$	$O(n)$
- insertar	$\Theta(\log n)$	$O(n)$
- Eliminar	$\Theta(\log n)$	$O(n)$

Buscar = Encontrar un valor específico dentro del árbol.

insertar = Agregar un nuevo valor al árbol manteniendo su estructura ordenada.

Eliminar = Quitar un valor existente del árbol sin romper su estructura.

• $O(\log n)$ = Esto significa que el tiempo de ejecución crece logaritmicamente con respecto al tamaño del árbol (n). A medida que el árbol se duplica en tamaño, el tiempo de busqueda, inserción o eliminación aumenta solo en una constante.

• $O(n)$ = indica que el tiempo de ejecución crece linealmente con respecto al tamaño del árbol.

En el peor de los casos, para un árbol desbalanceado, podría ser necesario recorrer todos los nodos del árbol, lo que resulta en un tiempo de ejecución proporcional al número de nodos.

• Average: En la mayoría de los casos, para un árbol binario de búsqueda bien balanceado, las operaciones tienen un tiempo de ejecución promedio de $O(\log n)$.

• Worst Case: El peor tiempo de ejecución posible para la operación. En el peor de los casos, por ejemplo, si el árbol está muy desbalanceado y se convierte en una lista enlazada, las operaciones podrían tardar $O(n)$.

④ Los árboles binarios de búsqueda son eficientes para muchas operaciones, pero su rendimiento depende en gran medida de su balance.

⑤ Si el árbol se vuelve muy desbalanceado, su rendimiento puede degradarse significativamente.

⑥ Hay nodos llamados "hojas". Son los nodos sin hijos, es decir, los que se encuentran al final de la estructura del árbol.

* Quiz: ¿Dónde se encuentra el nodo raíz?

• En la parte superior del árbol.

¿Cuál es la definición de nodo hoja?

• Es un nodo sin hijos.

¿Cómo se puede hacer que un árbol de búsqueda binaria se ejecute en tiempo $O(n)$?

• hacer que se inserten en Orden Ascendente y descendente.

Árboles transversales =

Los árboles transversales son una forma de recorrer y procesar los nodos de un árbol binario.

3 Formas = • In Orden, • Pre Orden, • Post Orden.

• In Orden = Izquierda, Root, Derecha

• Pre Orden = Root, Izquierda, Derecha

• Post orden = Izquierda, Derecha, Root.

tipos de recorrido =

A) In Orden = Se visitan primero los nodos del subárbol izquierdo, luego la raíz y finalmente los nodos del subárbol derecho.

- B) Pre orden = En este caso, se visita primero lo raíz (Root), luego el subarbol izquierdo y finalmente el subarbol derecho.

- C) Post orden = En este recorrido se visitan primero los nodos del subarbol izquierdo, luego los del subarbol derecho y finalmente lo raíz.

heaps =

Un heap es una estructura de datos especial que se asemeja a un árbol binario.

características =

- Heap Max: Cada nodo padre tiene un valor mayor o igual que sus hijos.
- Heap Min: Cada nodo padre tiene un valor menor o igual que sus hijos.

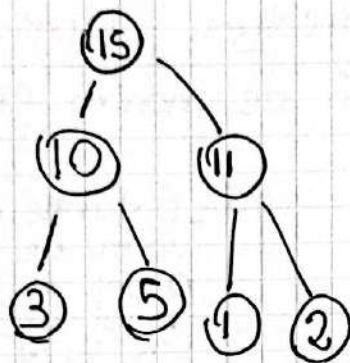
⊛ operaciones =

- A). Prioridad Queue = Los Heaps son ideales para implementar ^(Queue) colas de prioridad, donde los elementos se extraen en un orden específico basado en su prioridad.
- B). Ordenamiento: Algoritmo como Heapsort utilizan Heaps para ordenar elementos de manera eficiente.
- C). Algoritmos de búsqueda: Algunos algoritmos de búsqueda, como el algoritmo Dijkstra para encontrar el camino más corto en un grafo, utilizan Heaps para mantener un conjunto de candidatos en orden de prioridad.

D). Estructuras de datos más complejas =
Los heaps son componentes fundamentales
en la construcción de otras estructuras
de datos, como los tablos de hash y
los arboles binarios.

heap Ejemplo:

5, 3, 2, 10, 11, 1, 15



⊛ Previo al Orbol utilizomys el metodo de inter cambios (5 y 10), (10 y 3) etc. Este metodo se lo conoce como insertion.

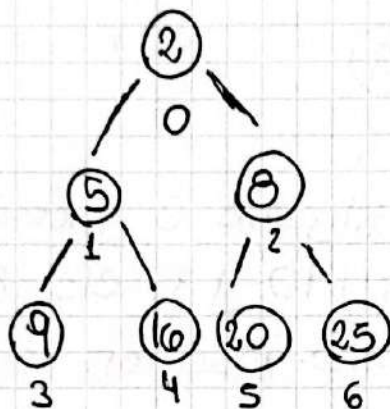
- insertion: Se agrega un nuevo elemento al final del orbol y luego se "Burbujea hacia arriba" intercambiandolo con su padre si es necesario, hasta que se restaura la propiedad del Heap.
 $O(\log n)$.

- check max: El check max es una situacion donde se visualizo el maximo (siempre en la parte superior). $O(1)$ constante.

⑨ Notas de lección:

- Lo propiedad de heap (monton) es el orden entre padres e hijos, lo que permite encontrar rápidamente el elemento máximo o mínimo y realizar operaciones de inserción y extracción de manera eficiente.

min heap:

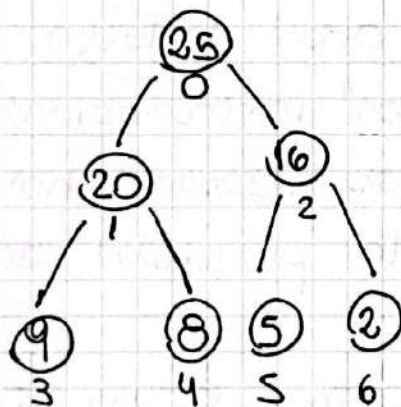


• Orden de los nodos: El valor de cada nodo padre es menor o igual que el valor de sus hijos.

• Root: El elemento de menor valor se encuentra en la raíz del árbol.

• Forma: Al igual que el árbol de max heap.

max heap:



• Orden de los nodos: El valor de cada nodo padre es mayor o igual que el valor de sus hijos.

• Root: El elemento de mayor valor se encuentra en la raíz del árbol.

• Forma: Todos los niveles están llenos excepto el último, se llena de izquierda a derecha.

Introducción Grafos:

Un Grafo es una estructura de datos abstracta que consiste en un conjunto de nodos (o vértices) conectados por aristas. Los nodos representan entidades y las aristas representan relaciones entre esas entidades.

Ejemplo:

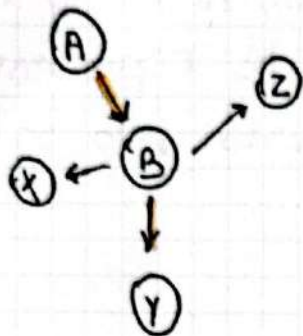
- Redes Sociales = Los usuarios son nodos y las conexiones de amistad son aristas.
- Mapas = Las ciudades son nodos y las carreteras son aristas.

Elementos de un Grafo:

- Nodos (o vértices) = Representan los elementos individuales del grafo.
- Aristas = Conectan los nodos y representan las relaciones entre ellos. Las aristas pueden ser:
 - Dirigidas = indican una dirección (una calle de un solo sentido).
 - No Dirigidas = no tienen dirección (una amistad).
 - Ponderados = tienen un valor asociado (distancia entre)

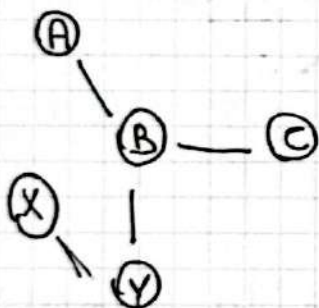
Tipos de Grafos:

A). Grafos dirigidos =



En el grafo dirigido los aristas tienen una dirección. (A, B, Y).

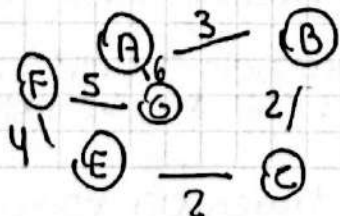
B). Grafos no Dirigidos =



En el grafo no dirigido los aristas no tienen una dirección.

C). Grafos Ponderados =

En el grafo Ponderado los aristas tienen un valor asociado.



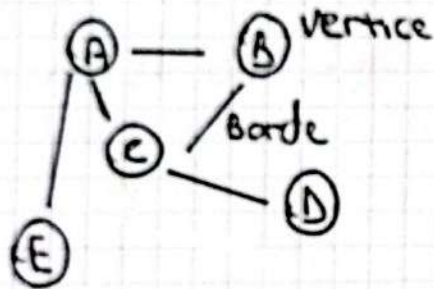
A, B, C = 5 Segmentos.

Terminología Grafos

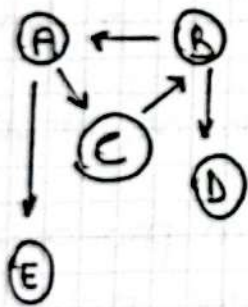
A) Vertice = $\{A, B, C, D, E\}$
Conjunto

B) Borde (edge) = $\{(A, B), (A, C), (B, C), (C, D), (A, E)\}$

Grafico =

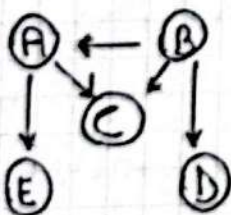


• Ciclico = Un GRFO ciclico, es ciclico si existe al

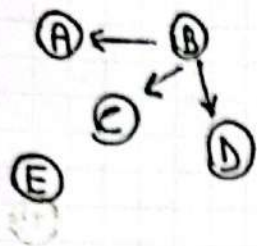


menos un camino cerrado (A, C, B) , es decir, una secuencia de vértices que comienza y termina en el mismo vértice.

• Acíclico = Un GRFO es acíclico si no contiene ningún ciclo. Los caminos son lineales, sin nombramientos que se cierran en sí mismos.



• Desconectado = Un Grafo Desconectado es desconectado si existen al menos dos vértices que no están conectados por ningún camino.



Ejemplo = Dos islas separadas por un océano. No hay forma de llegar de una a otra sin un medio de transporte que cruce el agua.

⊛ Existen grupos de vértices que no están relacionados entre sí.

Depth-First Search (DFS) = Búsqueda en Profundidad, es un algoritmo para recorrer y buscar en estructuras de datos como árboles o grafos.

• Como Funciona = * (utilizamos el método de Stack (pilas))

- 1 = inicio = se elige un nodo raíz como pto de partida.
- 2 = Exploración = Se explora a fondo una rama desde el nodo raíz, visitando todos los nodos de esa rama hasta llegar a un nodo sin hijos. (nodo hoja)
- 3 = Retroceso = Si se llega a un nodo hoja o se completa una rama, el algoritmo retrocede al nodo padre y explora otras ramas.

- 4. Repetición: Se repiten los pasos 2 y 3 hasta que se hayan visitado todos los nodos del árbol o Grafo.

Breadth First Search: o búsqueda en amplitud, es otro algoritmo para recorrer o buscar en estructuras de datos como árboles o grafos.

Como Funciona =

- 1. inicio: Se elige un nodo raíz como punto de partida.
- 2. Exploración: Se visitan todos los nodos vecinos (hijos) del nodo raíz.
- 3. niveles: Se exploran los nodos vecinos en el mismo orden en que fueron encontrados (FIFO = primero en entrar, primero en salir).
- 4. Repetición: Se repite el proceso 2 y 3 para C/u de los nodos visitados, explorando sus vecinos no visitados.

* Diferencias entre BFS y DFS:

- DFS: Explora en profundidad uno a uno antes de pasar a otro.
- BFS: Explora todos los nodos a un nivel antes de pasar al siguiente nivel.

hashing = es un proceso que consiste en transformar cualquier cantidad de datos (un texto, un archivo, una contraseña, etc), en una cadena de caracteres de longitud fija, conocido como valor hash o resumen criptográfico.

Esta cadena es única para esos datos específicos y es imposible de revertir para obtener los datos originales.

Colisiones =

- ocurren cuando dos o más datos diferentes generan el mismo valor hash.
- Apuntan a la misma posición en la tabla hash.

Open Addressing y Closed Addressing (Formas de manejar colisiones)

- O.A. = Cuando hay una colisión, se busca una nueva posición disponible en la tabla hash (busqueda lineal, cuadrático, etc).
- C.A. = Se utiliza una estructura de datos auxiliar (Lista Enlazada) para almacenar los elementos que colisionan en la misma posición.

Load Balance :

- Se refiere al punto en el que la tabla hash se vuelve demasiado llena, y el número de colisiones empieza a afectar significativamente el rendimiento.
- Es necesario aumentar el tamaño de la tabla hash para reducir el número de colisiones y mejorar la eficiencia.

* El Load Balance es un concepto importante para garantizar que la tabla hash siga siendo eficiente a medida que se añaden más datos.

Linear Probing (Sondeo Lineal) =

* El Sondeo Lineal es una técnica utilizada en las tablas hash para resolver colisiones. Cuando dos o más elementos generan el mismo índice en una tabla hash (colisión), el Sondeo Lineal nos proporciona una estrategia para encontrar una ubicación alternativa para almacenar el elemento.

Ejemplo = imagino un tabl hash de tamaño 10.
Si los elementos 'A' y 'B' generan el
índice 3, y el índice 3 ya está ocupado por A:

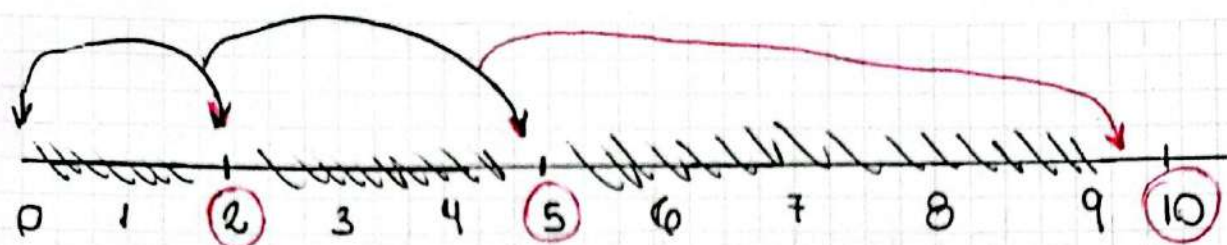
			A	C	B				
0	1	2	3	4	5	6	7	8	9

. Sondeo lineal = Se verifica el índice 4.
Si está ocupado, se pasará al índice
5, y así sucesivamente, hasta
encontrar una celda vacía.

* Desventaja = Si varios elementos colisionan
en el mismo índice o índices
cerca de ellos se forman grupos de
celdas ocupadas consecutivas,
lo que ralentiza las búsquedas.
 $O(n) \rightarrow O(1)$. (Agrupamiento
primario)

Sondeo Cuadrático =

Es una estrategia de resolución de colisiones
en tablas hash que, en lugar de buscar la
siguiente posición lineal, utiliza una secuencia
de índices calculados con una Función Cuadrática.
Esto significa que cada intento de encontrar
una celda vacía, se suma un num. creciente
obtenido elevando un contador al cuadrado.



$$* 1 + 1 \cdot 1 = 2$$

$$* 1 + 2 \cdot 2 = 5$$

$$* 1 + 3 \cdot 3 = 10$$

• Ejemplo:

Supongamos que tenemos un table hash y queremos insertar un elemento. Calculamos el índice inicial usando la Función hash. Si esa posición está ocupada.

1 = Sumamos 1^2 (es 1) al índice inicial y comprobamos esa nueva posición. $(1 + 1 \cdot 1)$

2 = Sumamos 2^2 (es 4) al índice inicial y comprobamos esa nueva posición. $(1 + 2 \cdot 2)$

3 = Sumamos 3^2 (es 9) al índice inicial y comprobamos esa nueva posición. $(1 + 3 \cdot 3)$

• Ventajas =

- menor agrupamiento = Al saltar posiciones de forma cuadrática, se reduce el riesgo de formar grupos de elementos colisionales.
- Elementos más uniforme: Los elementos tienden a dispersarse mejor.

Desventajas:

Podría llevar a un busqueda infinito si la tabla está muy llena.

Rehashing =

El Rehashing es el proceso de crear un nuevo table hash más grande y redistribuir todos los elementos de la table antigua a la nueva. Al aumentar el tamaño de la table, se reduce la probabilidad de colisiones y se mejora el rendimiento.

¿Cómo Funciona?

1. Crear un nuevo table = Se crea un nuevo table hash con un tamaño mayor al de la table actual.
2. Recalcular los índices = Para calcular se utiliza el índice hash, utilizando la nueva Función hash y el tamaño de la nuevo table.
3. Reinsertar los elementos = Cada elemento se inserta en su nueva posición en la table más grande.
4. Eliminar la table antigua (Una vez que los elementos fueron reubicados, la table antigua se descarta).